作品编号： CACR2025M2YPVW

作品类别： ☑软件设计 ☐硬件制作 ☐工程实践

☐密码应用技术 ☐其它

2025 年第十届全国密码技术竞赛作品设计报告

作品题目：针对缓存攻击的国产开源库的防护设计与评估

2025 年 10 月 23 日

中国密码学会

基本信息表
编号：CACR2025M2YPVW
作品题目：针对缓存攻击的国产开源库的防护设计与评估
作品类别：☑软件设计　☐硬件制作　☐工程实践　☐密码技术应用　☐其它

作品内容摘要：

开源密码库的广泛应用使其成为攻击者的核心攻击目标，在众多攻击手段中 cache 攻击是最具威胁的攻击类型之一。对于此类攻击，国际主流开源库已有成熟的防护体系，但国内密码库目前仍然存在防护短板，缺乏有效的系统化的 cache 攻击防护机制，使得国产库在关键使用场景中存在安全隐患。

本作品《针对缓存攻击的国产开源库的防护设计与评估》聚焦国产开源库 openHiTLS，结合国际上广泛使用的 OpenSSL 和 Crypto++开源库，分析 AES 和 SM4 两类分组密码算法在缓存侧信道攻击下的漏洞。通过对现有防护方案的分析与创新设计，本研究提出了多种针对性的防护措施，并在实际开源库中进行了验证与评估。

本系统提供了图形化操作界面，无需复杂配置，用户可以方便地选择测试对象、测试用例并执行性能测试、安全性测试。系统还支持测试结果对比功能，用户可以直观地查看不同防护策略下的性能差异与安全提升。此外，平台还允许用户上传自己的程序代码作为对象进行测试。

本研究的创新点主要体现为两个方面：一是防护部分的并行拆分表防护策略，该策略通过将查表操作拆分为多个并行子访问并重构合并结果，降低了缓存访问与密钥比特之间的相关性，从而以可接受的性能下降有效提升了防缓存攻击的安全性。且相比于传统策略，具有密钥与缓存访问相关性低、代码复杂度低、优化潜力大、易于分析等优势；二是安全检测部分，我们提出了新的、更易实施的安全评估方案，同时改进了既有的 Mastik 评估工具以适配多种对称加密算法的安全性评估。此外，我们针对 openHiTLS 库开发了通用的测试与评估平台 CacheSCA-Tool，用以验证和测试不同的安全防护方案的性能与安全性。

本作品为国产开源库的安全防护提供了可复用的平台，具备良好的实用价值与广泛的应用前景。尤其在高安全性要求的密码软件领域，具有重要参考意义和应用潜力。

关键词（五个）：防护加固，缓存攻击，分组密码，openHiTLS，并行拆分表

目录

1. 作品功能与性能说明 5
1.1 背景分析 5
1.2 作品功能介绍 7
1.2.1 防护加固模块 10
1.2.2 攻击检测模块 11
1.3 关键技术 12
1.3.1 缓存内攻击原理 12
1.3.2 分组密码介绍 14
1.3.3 脆弱性分析 19
2. openHiTLS 的安全防护与测试平台 21
2.1 平台组成简介 21
2.2 安全防护方案 21
2.2.1 已有防护方法 21
2.2.2 并行拆分表 24
2.3 防护方案实现 26
2.3.1 常量时间实现 26
2.3.2 预加载表实现 28
2.3.3 并行拆分表 32
2.3.4 三种防护方案的理论对比分析 35
2.4 检测方案与实现 37
2.4.1 检测方案介绍 37

2.4.2 正确性与性能测试程序 38
3. 系统测试与结果 41
3.1 实验环境 41
3.2 安全性测试 41
3.2.1 AES 41
3.2.2 SM4 44
3.2.3 总结 45
3.2.4 数据分析 46
3.3 性能测试 48
4. 应用前景 50
5. 结论 52

1. 作品功能与性能说明

1.1 背景分析

密码技术作为保障数据机密性、完整性与可用性的核心支撑，是数字经济时代网络安全的基石，其可靠实现直接关系到金融交易、政务通信等关键场景的安全稳定。而开源密码库是将密码算法、协议逻辑转化为可复用、可审计的开源代码集合，为软硬件开发者提供标准化的加密能力接口，是密码技术规模化落地的核心载体。在实际应用中，开源密码库的存在具有不可替代的必要性：一方面，其开源特性实现了代码透明度与社区审计的双重保障，能通过全球开发者协作快速发现并修复安全漏洞，避免闭源方案“黑箱风险”；另一方面，它大幅降低了密码技术的应用门槛——中小企业与开发者无需投入巨额成本自研加密模块，仅需通过调用库接口即可实现合规加密功能。从行业实践来看，开源密码库已成为网络基础设施的“隐形支柱”，支撑着全球主流 Web 服务器加密、VPN 通信及嵌入式设备安全功能[1]。

国外开源密码库的发展以 OpenSSL 为核心原点，后续分支均源于“特定场景需求与原库功能不匹配”的痛点，形成了“通用基础库 + 场景专用分支”的生态格局：1998 年基于早期 SSLeay 库重构的 OpenSSL 问世，以“跨平台兼容、算法全面、协议完整”的优势，逐步成为全球通用标准，在“库与组件”品类中占据重要市场份额，据统计其拥有超 16 万个客户，覆盖多个国家，支撑着 Apache、Nginx 等主流 Web 服务器的 HTTPS 加密功能。但其为兼容多场景导致代码冗余、API 设计复杂，为后续分支发展埋下伏笔。2006 年后场景专用库分化涌现。其年诞生的 mbed TLS 针对嵌入式设备资源受限问题，以“轻量级、模块化”设计填补 IoT 场景空白；2014 年 Google 主导的 BoringSSL 通过裁剪冗余代码，适配 Chrome 与 Android 的轻量化需求，LibreSSL 则聚焦代码精简与安全审计以规避 OpenSSL 的漏洞风险；2016 年 Google Tink 解决了多语言开发的加密接口统一问题，后续 OpenFHE、SEAL 等库则响应隐私计算需求，实现同态加密能力。这些库共同构成了覆盖全场景的国际开源密码生态[2]。

国际开源库虽功能成熟，但无法满足国内特殊需求：一是合规适配缺失，国外库未原生支持国密算法与协议，改造版本存在架构束缚；二是安全自主风险，核心代码与标准制定由国外主导，在国际技术竞争背景下存在“卡脖子”隐患；

三是场景适配不足，难以匹配国内物联网终端、政务云等场景的轻量、高安全需求。因此，构建国产开源库是保障国家信息安全战略、支撑数字经济自主发展的必然选择。所以，国内开源密码库的崛起是国家信息安全战略、密码合规要求、国内场景需求三重驱动的结果。国内开源库核心围绕 “国密算法支持” 展开，发展路径从“改造国外库”逐步走向“自主研发”：（1）早期改造型库快速满足国密需求。2014 年北京大学主导的 GmSSL 基于 OpenSSL 分支开发，通过新增 SM2/SM3/SM4 等国密算法与 TLCP 协议，解决了 OpenSSL 不支持国密标准的问题；2017 年江南天安开源的 TaSSL 则进一步优化国密适配，支持双证书体系与硬件密钥存储，适配政务、金融等合规场景[2]。这类库虽快速填补了国密空白，但受限于 OpenSSL 底层架构，难以满足定制化需求。（2）自主研发型库 openHITLS 的创新突破。2023 年，华为在全联接大会“开源创新，共筑智能世界”论坛上正式发布国内首个全面自研的开源密码套件 openHiTLS。密码应用超速发展，性能、资源、软硬件生态成为最迫切业务诉求，面向密码应用未来创新场景，国内首个全面自研密码套件 openHiTLS 应运而生。助推商密创新场景蓬勃发展，探索后量子创新实践，提供商密全栈加速、高安算法开箱即用、全场景极致轻量定制等企业级密码开源一站式解决方案[3]。这一举措既是响应《密码法》对关键领域自主密码技术的要求，也是填补国内全面自研密码套件空白的关键布局。

开源密码库的广泛应用使其成为攻击者的核心目标，攻击手段已从传统的漏洞利用升级为侧信道攻击等精细化方式，其中 cache 攻击因“低门槛、高隐蔽性”成为最具威胁的攻击类型之一。cache 攻击利用现代处理器的缓存机制，通过分析算法执行时的缓存访问模式，如命中/未命中的时间差异，逆向还原密钥等敏感信息，其危害体现在两个层面：

（1）攻击隐蔽性强，难以察觉与传统漏洞攻击不同，cache 攻击无需利用代码缺陷，仅通过监控缓存行为即可实施，不产生异常日志，常规安全防护手段难以检测。例如针对 AES 算法的 Flush+Reload 攻击，可在不干扰目标系统运行的情况下提取密钥，且攻击代码可通过共享内存跨进程、跨虚拟机执行。

（2）攻击技术成熟，门槛持续降低。目前已形成 Flush+Reload、Prime+Probe 等标准化攻击流程，且有 CacheKit、SideChannelAttackFramework 等开源工具支持，攻击者无需深厚密码学知识即可实施攻击，进一步放大了安全风险。

国际主流开源库已形成成熟的 cache 攻击防护体系，以 OpenSSL 为代表：采用恒定时间设计重构核心算法，消除数据依赖的分支与内存访问，使执行时间与敏感信息无关；优化内存布局与编译流程，通过缓存行对齐设计减少访问波动，集成-fconstant-time 等专用编译选项；建立自动化验证体系，将侧信道测试纳入 CI/CD 流程，确保版本迭代中防护效果不退化。此外，BoringSSL、LibreSSL 等分支也基于场景需求优化了防护策略，如针对嵌入式设备简化防护逻辑以平衡性能与安全。而国内开源库目前仍存在明显的 cache 攻击防护短板：早期改造型库如 GmSSL、TaSSL 的核心防护逻辑沿用 OpenSSL 框架，未针对 SM4 等国密算法进行优化。即使是自主研发的 openHiTLS，虽提及通过硬件加速手段降低风险，但尚未构建系统化的 cache 攻击防护机制。这种短板导致国产库在关键场景应用中存在安全隐患，难以满足《信息安全技术 密码模块安全技术要求》中抗侧信道攻击的等级标准。因此，本研究针对 openHiTLS 开展 cache 攻击防护设计与评估，具有明确的现实意义：既是填补国产开源密码库侧信道防护空白的技术突破，也是保障国密算法安全落地的关键举措，更能为自主开源密码生态的安全建设提供可复用的实践范本。

针对当前国产开源密码库在 cache 攻击防护领域的短板，本作品聚焦 openHiTLS 的安全能力升级，围绕“防护加固”与“评测”两大核心层次展开研究，形成“设计 -> 验证”的完整技术闭环。在防护加固设计层面，作品突破了直接复用国际成熟方案的局限。首先对国际主流的各种 cache 攻击防护策略进行系统性梳理与对比分析，接着在此基础上，提出我们自己的防护思想。在专项评测验证层面，作品构建了覆盖国际与国产算法的 cache 攻击评测体系，确保防护效果可量化、可验证。评测以 Flush+Reload、Prime+Probe 两种典型 cache 攻击为核心手段，分别对 openHiTLS 中原生未防护、采用国际通用防护及本作品提出的优化防护三种模式下的 AES、SM4 算法进行测试。

1.2 作品功能介绍

本作品名称为《针对缓存攻击的国产开源库的防护设计与评估》，主要针对国产开源库中的 openHiTLS。作品整体由两个核心模块构成：防护加固模块与攻击检测模块。防护加固模块负责在国产开源密码库 openHiTLS 中实现多种针对

缓存侧信道攻击的防护策略，并提供性能评估功能；攻击检测模块则通过扩展与改进 Mastik 攻击框架，实现对 openHiTLS 中算法实现的安全性评估与攻击效果验证。两者形成“防护—检测—评估”的完整闭环：防护模块提供多种防护策略，检测模块从攻击角度进行安全性验证，再通过性能与攻击时间等指标进行综合评估。整体系统可用于国产密码库侧信道防护机制的设计、验证、性能比较与安全评估，为后续国产密码软件的安全防护与优化提供可直接复用的平台。

本系统在实现功能模块之外，还提供了图形化的可视化操作界面，用于便捷地选择已编译的库文件、配置测试用例并执行正确性、性能与安全性测试，从而实现“防护—检测—评估”的一体化验收流程。界面采用多页布局：

主界面“Home”：选择测试对象。用户在该的“Target”选项中选择测试对象。系统默认提供四种由本项目编译出的库文件：AES,SM4 算法源代码（original）加上三种防护策略编译出的库——常量时间（constant_time）、预加载表（preload）、并行拆分表（lut_p）。用户也可导入并选择自己修改并编译的 openHiTLS 库文件（custom），系统对任意符合接口约定的库文件均可执行后续测试。选择了测试对象后，在“Cipher”选项中选择算法类型，系统现提供 AES，SM4 两种分组密码算法。

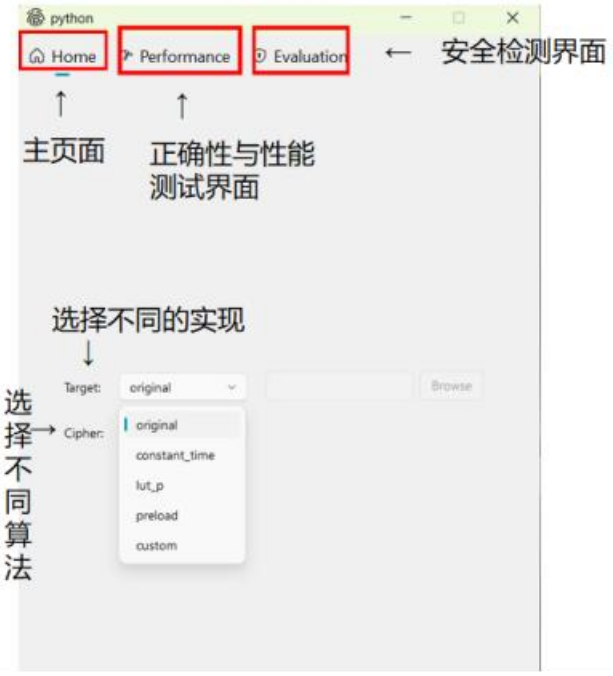

图 1-1 图形用户界面-主界面

性能与正确性测试界面“Performance”：用于验证防护后实现的功能正确性与测量性能开销，系统为性能/正确性测试定义了三种运行模式（test0,test1,test2）。测试界面通过“Save as ...”键支持将每次运行的测试结果保存，供后续对比使用。当用户在主界面选择了某个防护后库文件（例如 AES 的并行拆分表版本）并在测试页面点击“Test”运行后，可在“select for comparation”选项中选择同算法的其他已编译库文件的性能测试结果（例如常量时间版本、预加载表版本）或先前实验后保存的测试结果，可以选若干个测试结果用来对比。系统将自动将当前库文件的测试结果与用户选择的若干种测试结果并列对比，输出柱状图以直观展示性能差异对比。柱状图横轴为四种不同的负载情况，纵轴是时间（CPU 周期数）。

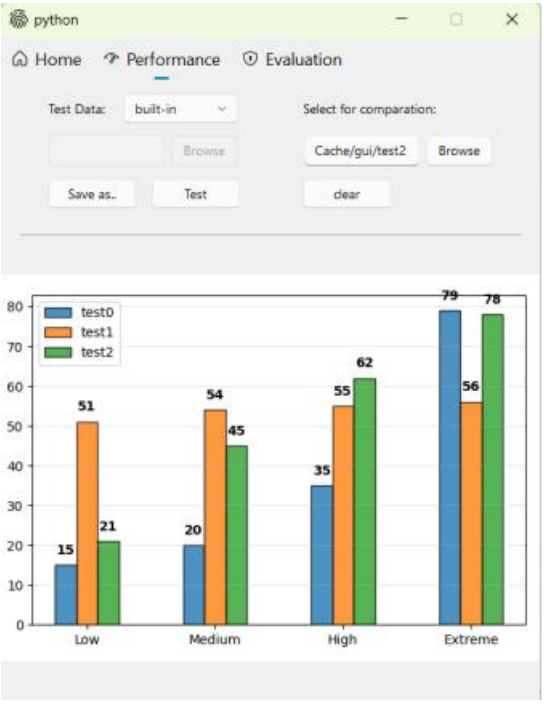

图 1-2 图形用户界面-性能与正确性测试界面

安全检测界面“Evaluation”：用于发起缓存攻击实验（支持 Flush+Reload 与 Prime+Probe），用户可在该页面输入待攻击的密钥、选择样本数量，系统将采集缓存访问时间并展示热力图与恢复密钥的尝试结果。

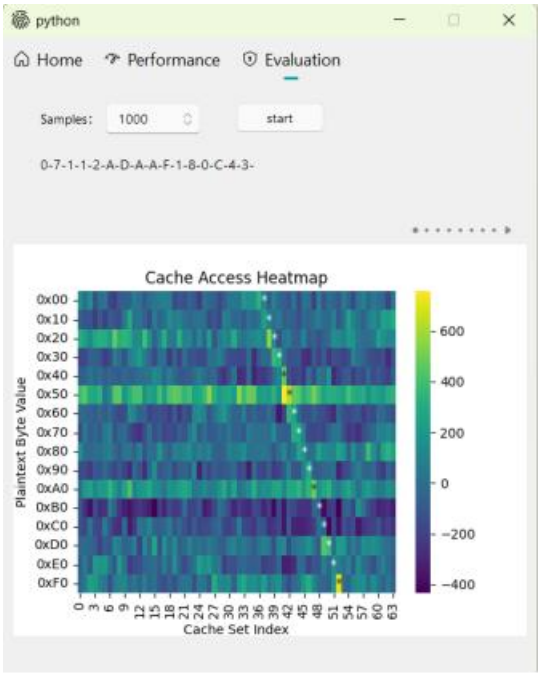

图 1-3 图形用户界面-安全检测界面

1.2.1 防护加固模块

防护加固模块针对国际与国内标准的分组密码算法 AES 与 SM4 在国产开源密码库 openHiTLS 中的实现进行安全加固。AES 是国际上应用最广泛的对称加密标准，而 SM4 是我国国家商用密码算法标准，二者均存在潜在的缓存访问相关泄漏风险。

在防护设计上，我们不仅借鉴了现有国际主流开源库的防护思路，如 OpenSSL 的常量时间策略（通过恒定时间消除数据依赖）与 Crypto++ 的预加载表策略（通过缓存预热减弱访问特征），还提出并实现了全新的自主防护策略——“并行拆分表策略”。 该策略是本作品的核心创新点：通过将查表操作拆分为多个并行子访问并重构合并结果，有效降低了缓存访问与密钥比特的相关性，从而提升防侧信道攻击的安全性，同时保持较高的执行效率。

在实现层面，我们在 openHiTLS 库中完成了上述三类防护的工程实现，并通过统一接口保证算法兼容性与可切换性。在性能评估方面，我们在相同平台下对三种不同防护策略的运行时间等指标进行了系统对比，验证了并行拆分表策略在安全性与性能平衡性上的优势。

防护模块的主要指标包括：

（1）防护算法覆盖：对 AES 与 SM4 两种标准算法进行防护；

（2）防护策略类型：实现三类防护（常量时间、预加载表、并行拆分表策略）；

（3）实现与验证：在 openHiTLS 中实现并测试全部三类防护；

（4）性能评估指标：比较不同策略下的加密耗时与性能开销。

1.2.2 攻击检测模块

攻击检测模块基于国际缓存攻击分析框架 Mastik 进行扩展与适配，构建了针对 openHiTLS 的攻击验证与安全评估系统。原始 Mastik 框架主要支持对 OpenSSL 实现的 AES 算法进行缓存攻击分析，且仅实现了单轮的密钥恢复，恢复密钥长度受限。

在本作品中，我们对 Mastik 进行了两方面的关键扩展：（1）二轮扩展：将原框架从单轮攻击扩展为双轮攻击流程，实现了完整的 AES/SM4 密钥恢复能力，使攻击检测更接近真实攻击强度；（2）算法与平台适配：针对 openHiTLS 库中 AES 与 SM4 的实现特性，改写了 Mastik 的缓存访问监控与模板匹配逻辑，使其能够正确地对国产开源库进行缓存攻击与数据采集。

模块可自动执行攻击过程、采集缓存访问时间、并统计分析攻击效率。我们通过对 AES 与 SM4 的多轮攻击实验，测量了攻击耗时等指标，定量评估了不同防护下的安全提升效果。

检测模块的主要指标包括：

（1）攻击类型覆盖：支持 Flush+Reload 与 Prime+Probe 两类攻击；

（2）算法覆盖：针对 AES 与 SM4 两种算法实现完整攻击测试；

（3）评估指标：对不同防护下的安全性（密钥恢复准确率）与性能（攻击时间）进行综合评估。

1.3 关键技术

1.3.1 缓存类攻击原理

缓存类攻击的核心是利用 CPU 缓存的“命中/未命中时间差异”构建侧信道，通过分析目标进程的缓存访问模式逆向敏感信息（如密码算法密钥）。目前针对密码库的主流缓存攻击手段为 Flush+Reload 与 Prime+Probe，二者均基于缓存分层结构与组相联映射机制实现，具体原理如下：

(1) 缓存结构与访问机制

1）缓存结构

现代计算机系统的存储体系呈现出典型的分层结构，从上到下依次为：CPU 寄存器 → L1 Cache → L2 Cache → L3 Cache → 主存→ 外存。这种层次结构体现了容量与访问速度之间的权衡关系：层级越靠上，容量越小但访问速度越快；层级越靠下，容量越大但访问速度越慢。当 CPU 访问数据时，会依次在寄存器、L1、L2、L3 Cache 中查找，若都未命中（Cache Miss），则访问主存甚至磁盘。

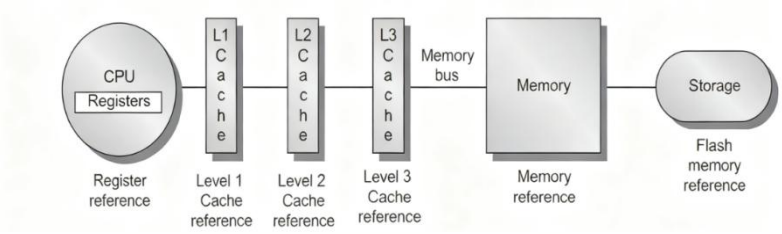

图 1-4　现代计算机系统的存储体系

Cache 分类：小于 4KB 的 Cache 多用于 L1 层；大于 4KB 的 Cache 多用于 L2/L3 层。

2）访问机制

Cache 的主要功能是加速 CPU 的数据访问过程。

CPU 地址由三部分组成：Address=[Tag｜Index｜Block Offset]。 Tag：标识主存块的唯一编号；Index：用于定位 Cache 中的组号；Block Offset：用于定位块内具体字节。

查找时，先通过 Index 定位组，再在组内比较 Tag。若匹配成功即为 Cache Hit；否则为 Cache Miss，需要从下一级加载数据。

在访问延迟上，L1 Cache 通常仅需约 3－5 个 CPU 周期，L2 约 10－20 个

周期，L3 约 30－50 个周期，而主存访问通常超过 100 个周期。因此，Cache 命中（Hit）能显著减少访问延迟[4]。

(2) 缓存攻击原理

缓存（cache）攻击利用处理器内存层次结构中不同访问延迟的差异来泄露受害进程的内存访问模式或秘密（例如加密算法的查表索引、分支行为等）。攻击者通过精心构造的内存访问与计时测量，不直接读取受害进程地址空间，而是通过观察缓存命中/未命中的时间差（通常以 CPU 周期为单位）推断受害者是否访问了某个缓存行或映射到某缓存组的地址。缓存攻击在多租户云环境、同一台机器上的非特权进程、甚至浏览器沙箱中均被证明是实用且高影响力的威胁。在研究中，最常见的两类缓存攻击分别是 Flush+Reload 与 Prime+Probe。

1）Flush+Reload

Flush + Reload 攻击中，攻击者进程需要与受害者进程共享同一块物理内存页。在 Flush 步骤中，攻击者先使用指令将被监控共享内存块驱逐出缓存。之后等待受害者执行。受害者访问共享内存块时，该内存块将加载进缓存中。之后执行 Reload 步骤，遍历被监控的共享内存块。其中，被受害者访问过的内存块因为加载到了缓存中，访问时间更短。由此，攻击者得以获知受害者访问的内存地址所对应的内存块。

攻击步骤：

Flush 阶段：攻击者使用 clflush 内在函数清空共享数组的缓存行。该指令强制将数据从所有缓存层级（L1、L2、L3）驱逐回主内存，确保后续访问必须从内存重新加载。

Trigger 阶段：受害者进程访问共享内存。由于缓存已被清空，受害者 CPU 需从主内存加载数据到其 L1 缓存。攻击者与受害者必须映射相同的物理内存页，例如通过共享库的.text 段。本例中 shared_array 模拟了这种共享。

Reload 阶段：攻击者重新读取该地址并使用高精度计时，如 rdtscp + lfence，测量访问延迟。若延迟低于校准阈值，判定为“命中”→ 受害者已访问；反之为“未命中”。通过对多个地址/时间窗口的重复采样，可以构建高信噪比的访存时间序列，用于密钥恢复或控制流推断。

2）Prime + Probe

Prime + Probe 攻击中，攻击者需要预先申请一个连续的字节数组作为驱逐集（evict set）。驱逐集大小与被利用的缓存大小相同。在 Prime 步骤中，将遍历驱逐集中的各内存块，从而驱逐出被利用缓存中的全部缓存行。之后将等待受害者执行。受害者访问内存块时，内存块映射到的缓存组将不得不驱逐出原先驱逐集填入的某条缓存行。之后执行 Prime 步骤，重新遍历驱逐集各内存块。此时受害者访问时驱逐出的缓存行所在的缓存集，将会有至少一个被映射到该缓存集上的驱逐集内存块在访问时发生缓存未命中，即访问时间较其它驱逐集内存块更长。因为该内存块在缓存中的对应缓存行被逐出，由此，攻击者得以获知受害者访问的内存地址的部分信息，即地址中的缓存组号。

攻击步骤：

Prime 阶段：攻击者访问一组地址，这些地址经过物理地址到 cache-set 的映射后，恰好填满目标缓存集合或若干集合。目的是在目标集合中放入攻击者已知的数据，使得任何受害者对同集合的访问将导致冲突并驱逐攻击者的数据。

Wait 阶段：攻击者暂停或转入睡眠，等待受害者运行其敏感代码。若受害者访问的内存地址与攻击者监控的某个缓存集映射到同一位置（通过地址映射计算确定），受害者的数据会加载到 L1 缓存，将攻击者填充的某些行驱逐。

Probe 阶段：攻击者重新访问相同的地址序列，并测量时间。若时间超过阈值（如 100 周期），表明数据不在 L1 缓存（需从 L2/L3 或内存加载），即缓存未命中→受害者访问过该集；若时间短，则缓存命中→受害者未访问。

1.3.2 分组密码介绍

(1) AES 算法

AES 是一种由美国国家标准与技术研究院（NIST）制定的对称分组加密算法，是目前全球应用最广泛的加密标准之一。AES 算法以 128 位为分组长度，支持 128/192/256 位密钥，加密过程含初始轮密钥加、N 轮迭代、最终轮三部分，每轮迭代包含四大操作（最后一轮只有 SubBytes, ShiftRows, AddRoundKey 操作）：

操作一。字节替代：将矩阵中的每个字节通过一个预先计算好的非线性替换表（S 盒）进行替换，这是算法非线性的主要来源，能有效抵抗密码分析。

操作二。行移位：将状态矩阵的每一行进行循环左移。第 0 行不移，第 1 行左移 1 字节，第 2 行左移 2 字节，第 3 行左移 3 字节。这个操作实现了行内的扩散。

操作三。列混淆：将状态矩阵的每一列视为一个多项式，在有限域 GF(2^8) 上与一个固定多项式进行模乘运算。这个线性变换提供了列内的扩散，使得一个字节的变化会影响整个列。

操作四。轮密钥加：将当前的状态矩阵与当前轮次生成的子密钥进行简单的按位异或操作。这是加密过程中唯一直接使用密钥的步骤[5]。

(2) SM4 算法

SM4 是中国自主设计的对称分组加密算法，是中国国家密码管理局（OSCCA）指定的商用密码标准。SM4 算法以 128 位为分组长度，使用 128 位密钥，加密过程共包含初始轮密钥加、32 轮迭代、最终轮 三部分。每轮迭代包含以下三个主要操作：

操作一。轮密钥加：将当前轮输入与对应的 32-bit 轮密钥 K_i 按位异或，形成轮函数输入。轮密钥加是唯一直接使用密钥的步骤，对整个加密安全性至关重要。SM4 的轮密钥加操作与 AES 类似，都是通过 XOR 实现状态与轮密钥的混合，但 SM4 的结构基于 32 位字的非线性函数 F，而非矩阵形式。

$$T_i = X_{i+1} \oplus X_{i+2} \oplus X_{i+3} \oplus K_i \quad \text{（公式 1.1）}$$

操作二。字节非线性替换：将上一步得到的 32-bit 输入 T_i 拆分为 4 个字节，对每个字节通过 8×8 S-box 表进行非线性替换。这是 SM4 的主要非线性来源，可抵抗线性和差分密码分析。AES 和 SM4 的 S 盒都是 8×8 位的非线性变换表，但是 AES 是基于 GF(2^8)上的逆元 + 仿射变换，而 SM4 是基于不同的固定查表映射。

$$B_i = S - box(T - i) \text{（公式 1.2）}$$

操作三。线性变换：对非线性替换后的 32-bit 输出 B_i 进行线性扩散，保证一位变化能影响整个 32-bit。线性变换公式（GF(2) 上的循环移位 + XOR）：

$$L(B_i) = B_i \oplus (B_i \ll 2) \oplus (B_i \ll 10) \oplus (B_i \ll 18) \oplus (B_i \ll 24) \text{（公式 1.3）}$$

输出结果再与 Xi 异或得到下一轮输入：

$$X_{i+4} = X_i \oplus L(B_i) \quad \text{（公式 1.4）}$$

(3) 非线性部件的主流软件实现方式

对称分组密码算法的安全性在很大程度上依赖于其非线性部件的设计与实现。AES 和 SM4 作为目前国际及国内广泛应用的两种分组加密标准，其非线性变换均由 S 盒实现，是算法抵抗线性与差分攻击的核心环节。在软件实现中，为了兼顾性能与安全，研究者提出了多种查表实现方式。下文将分别介绍 AES 与 SM4 的非线性部件的主流实现方法及其特点。

1）S 盒的实现

把 256 个输入对应的输出放成一个 uint8_t sbox[256]，运行时直接用字节值做索引读取。实现简单、速度快（尤其在没有硬件指令加速时），但索引访问会带来可被缓存侧信道利用的内存访问模式。

```
static const uint8_t AES_S[256] = {
    0x63U, 0x7cU, 0x77U, 0x7bU, 0xf2U, 0x6bU, 0x6fU, 0xc5U, 0x30U, 0x01U, 0x67U, 0x2bU, 0xfeU, 0xd7U, 0xabU, 0x76U,
    0xcaU, 0x82U, 0xc9U, 0x7dU, 0xfaU, 0x59U, 0x47U, 0xf0U, 0xadU, 0xd4U, 0xa2U, 0xafU, 0x9cU, 0xa4U, 0x72U, 0xc0U,
    0xb7U, 0xfdU, 0x93U, 0x26U, 0x36U, 0x3fU, 0xf7U, 0xccU, 0x34U, 0xa5U, 0xe5U, 0xf1U, 0x71U, 0xd8U, 0x31U, 0x15U,
    0x04U, 0xc7U, 0x23U, 0xc3U, 0x18U, 0x96U, 0x05U, 0x9aU, 0x07U, 0x12U, 0x80U, 0xe2U, 0xebU, 0x27U, 0xb2U, 0x75U,
    0x09U, 0x83U, 0x2cU, 0x1aU, 0x1bU, 0x6eU, 0x5aU, 0xa0U, 0x52U, 0x3bU, 0xd6U, 0xb3U, 0x29U, 0xe3U, 0x2fU, 0x84U,
    0x53U, 0xd1U, 0x00U, 0xedU, 0x20U, 0xfcU, 0xb1U, 0x5bU, 0x6aU, 0xcbU, 0xbeU, 0x39U, 0x4aU, 0x4cU, 0x58U, 0xcfU,
    0xd0U, 0xefU, 0xaaU, 0xfbU, 0x43U, 0x4dU, 0x33U, 0x85U, 0x45U, 0xf9U, 0x02U, 0x7fU, 0x50U, 0x3cU, 0x9fU, 0xa8U,
    0x51U, 0xa3U, 0x40U, 0x8fU, 0x92U, 0x9dU, 0x38U, 0xf5U, 0xbcU, 0xb6U, 0xdaU, 0x21U, 0x10U, 0xffU, 0xf3U, 0xd2U,
    0xcdU, 0x0cU, 0x13U, 0xecU, 0x5fU, 0x97U, 0x44U, 0x17U, 0xc4U, 0xa7U, 0x7eU, 0x3dU, 0x64U, 0x5dU, 0x19U, 0x73U,
    0x60U, 0x81U, 0x4fU, 0xdcU, 0x22U, 0x2aU, 0x90U, 0x88U, 0x46U, 0xeeU, 0xb8U, 0x14U, 0xdeU, 0x5eU, 0x0bU, 0xdbU,
    0xe0U, 0x32U, 0x3aU, 0x0aU, 0x49U, 0x06U, 0x24U, 0x5cU, 0xc2U, 0xd3U, 0xacU, 0x62U, 0x91U, 0x95U, 0xe4U, 0x79U,
    0xe7U, 0xc8U, 0x37U, 0x6dU, 0x8dU, 0xd5U, 0x4eU, 0xa9U, 0x6cU, 0x56U, 0xf4U, 0xeaU, 0x65U, 0x7aU, 0xaeU, 0x08U,
    0xbaU, 0x78U, 0x25U, 0x2eU, 0x1cU, 0xa6U, 0xb4U, 0xc6U, 0xe8U, 0xddU, 0x74U, 0x1fU, 0x4bU, 0xbdU, 0x8bU, 0x8aU,
    0x70U, 0x3eU, 0xb5U, 0x66U, 0x48U, 0x03U, 0xf6U, 0x0eU, 0x61U, 0x35U, 0x57U, 0xb9U, 0x86U, 0xc1U, 0x1dU, 0x9eU,
    0xe1U, 0xf8U, 0x98U, 0x11U, 0x69U, 0xd9U, 0x8eU, 0x94U, 0x9bU, 0x1eU, 0x87U, 0xe9U, 0xceU, 0x55U, 0x28U, 0xdfU,
    0x8cU, 0xa1U, 0x89U, 0x0dU, 0xbfU, 0xe6U, 0x42U, 0x68U, 0x41U, 0x99U, 0x2dU, 0x0fU, 0xb0U, 0x54U, 0xbbU, 0x16U
};
```

图 1-5　openHiTLS 开源库中 AES 算法中的 S 盒

```
15      static const uint8_t SM4_S[256] = {
16          0xD6, 0x90, 0xE9, 0xFE, 0xCC, 0xE1, 0x3D, 0xB7, 0x16, 0xB6, 0x14, 0xC2,
17          0x28, 0xFB, 0x2C, 0x05, 0x2B, 0x67, 0x9A, 0x76, 0x2A, 0xBE, 0x04, 0xC3,
18          0xAA, 0x44, 0x13, 0x26, 0x49, 0x86, 0x06, 0x99, 0x9C, 0x42, 0x50, 0xF4,
19          0x91, 0xEF, 0x98, 0x7A, 0x33, 0x54, 0x0B, 0x43, 0xED, 0xCF, 0xAC, 0x62,
20          0xE4, 0xB3, 0x1C, 0xA9, 0xC9, 0x08, 0xE8, 0x95, 0x80, 0xDF, 0x94, 0xFA,
21          0x75, 0x8F, 0x3F, 0xA6, 0x47, 0x07, 0xA7, 0xFC, 0xF3, 0x73, 0x17, 0xBA,
22          0x83, 0x59, 0x3C, 0x19, 0xE6, 0x85, 0x4F, 0xA8, 0x68, 0x6B, 0x81, 0xB2,
23          0x71, 0x64, 0xDA, 0x8B, 0xF8, 0xEB, 0x0F, 0x4B, 0x70, 0x56, 0x9D, 0x35,
24          0x1E, 0x24, 0x0E, 0x5E, 0x63, 0x58, 0xD1, 0xA2, 0x25, 0x22, 0x7C, 0x3B,
25          0x01, 0x21, 0x78, 0x87, 0xD4, 0x00, 0x46, 0x57, 0x9F, 0xD3, 0x27, 0x52,
26          0x4C, 0x36, 0x02, 0xE7, 0xA0, 0xC4, 0xC8, 0x9E, 0xEA, 0xBF, 0x8A, 0xD2,
27          0x40, 0xC7, 0x38, 0xB5, 0xA3, 0xF7, 0xF2, 0xCE, 0xF9, 0x61, 0x15, 0xA1,
28          0xE0, 0xAE, 0x5D, 0xA4, 0x9B, 0x34, 0x1A, 0x55, 0xAD, 0x93, 0x32, 0x30,
29          0xF5, 0x8C, 0xB1, 0xE3, 0x1D, 0xF6, 0xE2, 0x2E, 0x82, 0x66, 0xCA, 0x60,
30          0xC0, 0x29, 0x23, 0xAB, 0x0D, 0x53, 0x4E, 0x6F, 0xD5, 0xDB, 0x37, 0x45,
31          0xDE, 0xFD, 0x8E, 0x2F, 0x03, 0xFF, 0x6A, 0x72, 0x6D, 0x6C, 0x5B, 0x51,
32          0x8D, 0x1B, 0xAF, 0x92, 0xBB, 0xDD, 0xBC, 0x7F, 0x11, 0xD9, 0x5C, 0x41,
33          0x1F, 0x10, 0x5A, 0xD8, 0x0A, 0xC1, 0x31, 0x88, 0xA5, 0xCD, 0x7B, 0xBD,
34          0x2D, 0x74, 0xD0, 0x12, 0xB8, 0xE5, 0xB4, 0xB0, 0x89, 0x69, 0x97, 0x4A,
35          0x0C, 0x96, 0x77, 0x7E, 0x65, 0xB9, 0xF1, 0x09, 0xC5, 0x6E, 0xC6, 0x84,
36          0x18, 0xF0, 0x7D, 0xEC, 0x3A, 0xDC, 0x4D, 0x20, 0x79, 0xEE, 0x5F, 0x3E,
37          0xD7, 0xCB, 0x39, 0x48
38      };
```

图 1-6　OpenSSL 开源库中 SM4 算法中的 S 盒

2）T 表的实现

T-table 是一种软件优化策略，把一轮中多个子操作（如 SubBytes、ShiftRows、MixColumns）预先组合成对单字节的查表输出（通常 32-bit 或 64-bit），从而把原本需要多次查表/移位/GF 乘法的操作合并成少次内存加载 + 若干 XOR。在没有硬件加速（如 AES-NI）时，T-table 能显著提升 AES 在通用 CPU 上的吞吐。但是 T-table 的索引通常直接由明文字节/中间状态字节决定，导致巨大的缓存侧信道泄露风险（攻击者通过观察缓存命中/缺失或时间差异恢复密钥），因此在安全敏感场景应慎用或采取恒时实现/硬件指令替代。

在 AES 的主轮（非最后一轮）中，典型操作顺序为 SubBytes -> ShiftRows -> MixColumns -> AddRoundKey。可以将 SubBytes + MixColumns 对单字节 x 的复合结果预先计算为 32-bit 表项 T0[x]（4 字节），这样一列的输出可由 4 次查表 XOR 合成。通常实现会有 4 张表 T0..T3（每张 256×32bit，共 4KB）。

```
71      static const uint32_t TE0[256] = {
72          0xc66363a5U, 0xf87c7c84U, 0xee777799U, 0xf67b7b8dU,
73          0xfff2f20dU, 0xd66b6bbdU, 0xde6f6fb1U, 0x91c5c554U,
74          0x60303050U, 0x02010103U, 0xce6767a9U, 0x562b2b7dU,
75          0xe7fefe19U, 0xb5d7d762U, 0x4dababe6U, 0xec76769aU,
76          0x8fcaca45U, 0x1f82829dU, 0x89c9c940U, 0xfa7d7d87U,
77          0xeffafa15U, 0xb25959ebU, 0x8e4747c9U, 0xfbf0f00bU,
78          0x41adadecU, 0xb3d4d467U, 0x5fa2a2fdU, 0x45afafeaU,
79          0x239c9cbfU, 0x53a4a4f7U, 0xe4727296U, 0x9bc0c05bU,
80          0x75b7b7c2U, 0xe1fdfd1cU, 0x3d9393aeU, 0x4c26266aU,
81          0x6c36365aU, 0x7e3f3f41U, 0xf5f7f702U, 0x83cccc4fU,
82          0x6834345cU, 0x51a5a5f4U, 0xd1e5e534U, 0xf9f1f108U,
83          0xe2717193U, 0xabd8d873U, 0x62313153U, 0x2a15153fU,
84          0x0804040cU, 0x95c7c752U, 0x46232365U, 0x9dc3c35eU,
85          0x30181828U, 0x379696a1U, 0x0a05050fU, 0x2f9a9ab5U,
```

图 1-7　openHiTLS 开源库中 AES 算法中的 TE0 表

在 SM4 算法中，每一轮的核心操作包括字节替换和线性扩散。为了提升加密效率，软件实现中通常将这两步合并为一次查表操作，即构建 T 表。这种方法与 AES 的实现思路类似：通过预计算，将「S 盒替换 + 线性变换」的结果存入查找表中，轮函数时只需查表并进行若干次异或操作即可得到最终结果。SM4 的 T 表通常由四张 256×32bit 的表组成，分别对应输入字的四个字节位置（T0、T1、T2、T3）。

```
35      static const uint32_t XBOX_0[] = {
36          0xd55b5b8e, 0x924242d0, 0xeaa7a74d, 0xfdfbfb06, 0xcf3333fc, 0xe2878765, 0x3df4f4c9, 0xb5dede6b,
37          0x1658584e, 0xb4dada6e, 0x14505044, 0xc10b0bca, 0x28a0a088, 0xf8efef17, 0x2cb0b09c, 0x05141411,
38          0x2bacac87, 0x669d9dfb, 0x986a6af2, 0x77d9d9ae, 0x2aa8a882, 0xbcfafa46, 0x04101014, 0xc00f0fcf,
39          0xa8aaaa02, 0x45111154, 0x134c4c5f, 0x269898be, 0x4825256d, 0x841a1a9e, 0x0618181e, 0x9b6666fd,
40          0x9e7272ec, 0x4309094a, 0x51414110, 0xf7d3d324, 0x934646d5, 0xecbfbf53, 0x9a6262f8, 0x7be9e992,
41          0x33ccccff, 0x55515104, 0x0b2c2c27, 0x420d0d4f, 0xeeb7b759, 0xcc3f3ff3, 0xaeb2b21c, 0x638989ea,
42          0xe7939374, 0xb1cece7f, 0x1c70706c, 0xaba6a60d, 0xca2727ed, 0x08202028, 0xeba3a348, 0x975656c1,
43          0x82020280, 0xdc7f7fa3, 0x965252c4, 0xf9ebeb12, 0x74d5d5a1, 0x8d3e3eb3, 0x3ffcfcc3, 0xa49a9a3e,
44          0x461d1d5b, 0x071c1c1b, 0xa59e9e3b, 0xfff3f30c, 0xf0cfcf3f, 0x72cdcdbf, 0x175c5c4b, 0xb8eaea52,
45          0x810e0e8f, 0x5865653d, 0x3cf0f0cc, 0x1964647d, 0xe59b9b7e, 0x87161691, 0x4e3d3d73, 0xaaa2a208,
46          0x69a1a1c8, 0x6aadadc7, 0x83060685, 0xb0caca7a, 0x70c5c5b5, 0x659191f4, 0xd96b6bb2, 0x892e2ea7,
47          0xfbe3e318, 0xe8afaf47, 0x0f3c3c33, 0x4a2d2d67, 0x71c1c1b0, 0x5759590e, 0x9f7676e9, 0x35d4d4e1,
48          0x1e787866, 0x249090b4, 0x0e383836, 0x5f797926, 0x628d8def, 0x59616138, 0xd2474795, 0xa08a8a2a,
49          0x259494b1, 0x228888aa, 0x7df1f18c, 0x3bececd7, 0x01040405, 0x218484a5, 0x79e1e198, 0x851e1e9b,
50          0xd7535384, 0x00000000, 0x4719195e, 0x565d5d0b, 0x9d7e7ee3, 0xd04f4f9f, 0x279c9cbb, 0x5349491a,
51          0x4d31317c, 0x36d8d8ee, 0x0208080a, 0xe49f9f7b, 0xa2828220, 0xc71313d4, 0xcb2323e8, 0x9c7a7ae6,
52          0xe9abab42, 0xbdfefe43, 0x882a2aa2, 0xd14b4b9a, 0x41010140, 0xc41f1fdb, 0x38e0e0d8, 0xb7d6d661,
53          0xa18e8e2f, 0xf4dfdf2b, 0xf1cbcb3a, 0xcd3b3bf6, 0xfae7e71d, 0x608585e5, 0x15545441, 0xa3868625,
54          0xe3838360, 0xacbaba16, 0x5c757529, 0xa6929234, 0x996e6ef7, 0x34d0d0e4, 0x1a686872, 0x54555501,
```

图 1-8　openHiTLS 开源库中 SM4 算法中的 T 表

1.3.3 脆弱性分析

AES 与 SM4 的非线性变换都依赖字节级或字级的查表/预计算（S 盒或 T 表）来实现算法的非线性与部分线性组合。查表带来的高性能同时也引入了输入相关的内存访问模式：某一中间字节的取值直接决定了程序要访问表的哪一项或哪一组连续项（cache line）。现代处理器缓存的层次化与有限的行大小（典型 64 字节）使得“表项 → cache line”的映射显著降低了攻击者从时间测量中断定受害者访问哪一类表项的难度——这就是缓存侧信道攻击得以成功的根本原因。下面按要点总结脆弱性与攻击流程。

(1) 共同脆弱性点

表项地址依赖密钥/中间状态：T 表或 S 盒查表的索引通常是明文/中间状态的函数，而中间状态又与密钥直接相关，因此查表地址与密钥存在可利用的函数关系。

缓存行粒度泄露：一个典型缓存行（64B）可容纳多个相邻表项。因此攻击者通过判断某一缓存行是否被命中，可以将索引的候选集合缩小到行内项集合，从而显著降低暴力搜索空间。

多轮/多字节累积信息：在多次加密（或对不同明文）收集的观测中，可以统计并交叉参考不同明文下哪些表项/行被命中，从而合并约束逐字节恢复轮密钥或最终轮输入。

公共实现模式：大多数高性能软件实现（例如基于 T-table 的实现）在最后几轮仍使用查表（或 Te4/TeX），因此对最后一轮的监控极具价值 —— 可以直接映射为密钥推导公式从而恢复轮密钥并逆向出主密钥。

(2) 典型攻击流程

准备（共享内存 / 地址定位）：攻击者确保与受害者共享含有目标查表的物理页（例如映射相同库文件或利用页合并），并找出候选表地址范围。

定位活跃表项：对候选地址逐行执行 flush（clflush）→ 等待 victim → reload + 精确计时；若 reload 命中（延迟短），说明该行在 victim 执行期间被访问。遍历可定位到活跃表行集合。

针对性监控：缩小到关键表行（往往是包含最后一轮表项的一行），对大量

加密操作重复 flush→trigger→reload 测量，记录每次的计时与对应密文/明文。

统计与钥匙推导：利用加密最后一轮的简化代数关系，将命中/未命中结果映射为若干个候选中间字节值，结合知道的密文/明文可枚举并统计出现频率，最终确定正确字节并恢复轮密钥，继而逆推主密钥。

(3) SM4 与 AES 在攻击线索与恢复上的异同

表结构与映射差异：AES 的 T-table（Te0..Te3）典型为 256×32 位项，最后轮常用 Te4-like 表简化求解；SM4 的 T 表（L0..L3）每表 256 项、每项 32 位，且在每轮都参与计算 —— 因此对 SM4 来说每轮都可能泄露有关中间 32 位字的线索，攻击者可以利用多轮观测集合化约候选集。

密码结构差异对逆推的影响：AES 为 SubBytes+ShiftRows+MixColumns 的替换-置换结构，最后一轮有一定的可线性化形式使得通过查表获知 Te4 项后能直接推出最后一轮中间值和轮密钥；SM4 为非平衡 Feistel，轮函数为$X_{i+4} = X_i \oplus T(\cdot)$，T 的输出直接异或到下一轮字，因此获取某轮 T 表的被访问项即可直接约束对应的轮密钥候选，继而逆推前一轮。

轮数与复杂度：SM4 的轮数较多（32 轮）但攻击仍以最后轮或倒数若干轮为突破口；AES 的轮数较少（10/12/14），但 T-table 结构使最后一轮攻击通常更直接。

2. openHiTLS 的安全防护与测试平台

2.1 平台组成简介

本平台采用模块化设计，以“防护—检测—评估”的闭环流程为核心，面向对国产开源密码库 openHiTLS 进行缓存侧信道防护实现、攻击验证与性能安全评估。总体架构由四个逻辑层次构成：

实验/被测层：放置被测库（openHiTLS 的不同编译版本：三种防护实现），以标准化 API 提供加解密服务。被测代码供性能测试与攻击模块调用。

防护与评估层：包含防护加固实现（常量时间、预加载表、并行拆分表）与性能/正确性测试驱动。该层负责在相同硬件/编译条件下收集性能数据并对正确性进行断言。

攻击与检测层：基于扩展后的 Mastik 框架实现 Flush+Reload 与 Prime+Probe 采样器、模板匹配与密钥恢复流程。该层负责自动化发起攻击、采集时间序列与导出热力图。

可视化交互层：提供用户交互界面（测试对象选择、性能/正确性测试、攻击实验配置与结果展示）、实验管理与结果对比导出功能。GUI 与后端通过明确 API/文件格式交互，支持脚本化调用与批量实验。

2.2 安全防护方案

2.2.1 已有防护方法

(1) 调研范围与选型说明

在研究针对缓存攻击（cache attack）的防护机制时，本文主要聚焦于软件层面的实现与防护策略。当前主流的分组加密算法（如 AES、SM4）在开源密码库中广泛应用，其具体实现方式直接影响算法的性能与安全性。因此，为了分析不同防护手段在实际工程环境中的适用性与效果，本研究选择了具有代表性的开源加密库作为调研对象。

目前，国际上常用的开源密码库包括 OpenSSL、Crypto++、mbedTLS、Botan、LibreSSL 等。这些库在算法实现、硬件加速接口、侧信道防护程度及性能优化方向上各有差异。综合考量后，我们发现 OpenSSL 与 Crypto++更适合我们的研究需

要。OpenSSL 能反映真实生产环境中侧信道防护措施的部署与性能影响，Crypto++则适用于验证底层算法结构在不同防护机制下的安全与效率变化。通过对这两者的联合分析，可以在性能、安全性与工程可行性之间建立系统化的对比基础，从而形成具有普适意义的防护设计与评估结论。

1）OpenSSL：

OpenSSL 是目前全球应用最广泛的加密库之一，几乎被所有主流操作系统、网络安全协议与服务器框架所采用。例如，HTTPS（TLS/SSL）通信、SSH 安全连接、VPN 加密隧道、操作系统内核模块及云平台均依赖 OpenSSL 提供的加密基础。

OpenSSL 的 AES 与 SM4 实现具有多种版本选择，包括基于查表（T-table）的高速实现、基于 AES-NI 等硬件指令集的加速实现、以及常量时间的安全实现选项。因此，OpenSSL 既能代表高性能通用实现，又能覆盖抗侧信道防护实现的实际工程路径，适合用于不同防护级别下的性能与安全对比分析。

2）Crypto++：

Crypto++是一个以研究和教学为导向的跨平台密码库，特点是算法覆盖全面、模块化设计灵活，便于独立分析具体算法的实现细节。与 OpenSSL 相比，Crypto++的代码结构更简洁，算法边界清晰，适合对特定算法（如 AES）的实现细节与防护机制进行精确控制与修改。Crypto++提供了多种 AES 实现方案，包括传统查表法、预取防护实现，因此可以较好地反映不同软件防护策略在纯软件环境下的性能差异与安全性表现。

此外，Crypto++在学术界使用广泛，常作为算法验证与防护机制实验的参考框架，被多篇侧信道攻击与防护相关论文采用，如 CHES、TCHES 会议论文中的实验平台，因此具有较强的研究代表性。

(2) 防护策略选型与分析方法

在针对缓存侧信道攻击的防护研究中，软件层面的防护策略主要分为两类：一类以常量时间计算为核心，通过消除输入相关的内存访问与分支行为来彻底阻断攻击信号；另一类以缓存行为扰动或隐藏为目标，通过预加载降低攻击者观测的有效性。本文我们使用 OpenSSL 的常量时间(constant-time)实现和 crypto++的预加载表(reload)实现分别作为前面介绍的两类防护策略的典型代表。本文通过

源码分析、实验测试与性能评估，对两种防护策略的内部机制与防护效果进行系统对比。

1）OpenSSL 的常量时间防护策略

OpenSSL 在高安全场景下（如启用 OPENSSL_AES_CONST_TIME 宏）采用常量时间 AES 实现，其核心思想是通过算法级的逻辑化重构，使每一步运算的执行时间与输入数据完全无关，从根本上消除缓存访问差异。具体而言，OpenSSL 的常量时间实现摒弃了传统的查表（S-box 或 T-table）方式，而是将 S 盒运算分解为在有限域 $GF(2^8)$上的代数计算：包括乘法逆运算与仿射变换。这一过程仅由固定序列的逻辑运算（XOR、AND、SHIFT）构成，不涉及输入相关的内存访问或条件分支，因此在执行层面实现了真正的时间常数性[7]。

这种实现方式的理论基础在于“消除数据相关访存”原则。它牺牲部分性能以换取更强的防侧信道能力，是当前密码学软件实现中最安全、最彻底的防护手段之一。然而，由于完全基于逻辑运算的 S 盒计算比查表法复杂，常量时间实现的性能一般低于传统查表方式，但在高安全要求（如安全芯片、TLS 核心模块）中被广泛采用。

2）Crypto++ 的预加载表防护策略

Crypto++的防护策略以性能与安全的折中为出发点，采用了预加载表防护机制。该方法保留了高性能的查表（T-table 或 S-box）结构，但在加密开始前，会主动将整个查表内容加载到缓存中。这样，在后续的加密轮次中，无论输入数据为何，访问表项都能命中缓存，从而显著减弱输入相关的缓存访问时间差异[8]。

这种方法的设计思想虽然无法彻底消除输入相关访问，但可以将缓存状态标准化，使攻击者难以通过缓存命中时间推断具体访问内容。其核心优势在于实现简单、性能损耗极低，能够在不改动算法逻辑的情况下增强安全性。然而，该策略属于“部分防护”方案，主要针对 Flush+Reload 类型的攻击，对高精度或主动驱逐型攻击（如 Prime+Probe、Evict+Time）的防护能力有限。攻击者若能监测整个缓存集合的访问模式，仍可能从微小的命中差异中提取信息。

2.2.2 并行拆分表（Parallel Separation of Items）

设受害者的密钥为 k ∈ K，其随机变量为 K；受害者访问的内存地址为 a ∈ A，其随机变量为 A；受害者访问地址对应的缓存组号为 c ∈ C，其随机变量为 C；攻击者采集到的时间信息 t ∈ T，其随机变量为 T。设缓存组数为 S，关联度为 E，缓存行大小为 B（缓存行大小的单位为 Byte），且 $s=\log_2 S$, $e=\log_2 E$, $b=\log_2 B$。在本作品中，B=64Byte。

缓存组号 c 只由内存地址 a 的部分地址决定。在二进制表示下，c 是 a 的中间 s 位，且二者满足：

$$(c)_2 = ((a)_2 \gg b)\ mod\ S \text{（公式 2-1）}$$

因此，若 $\forall a \in A, \exists a_0 \in A, (a)_2 \gg b = (a_0)_2 \gg b$，则 $\forall c \in C, \exists c_0 \in C, (c)_2 = (c_0)_2$。故：

$$\forall c \in C, P(C = c) = \{1 \quad if\ c = c_0\text{，}0 \quad if\ c \neq c_0\} \text{（公式 2-2）}$$

进一步可得：

$$if\ c = c_0 : \forall a \in A,, P(A = a, C = c) = P(A = a)$$

$$if\ c \neq c_0 : \forall a \in A,, P(A = a, C = c) = P(\emptyset) = 0 \text{（公式 2-3）}$$

故此时 $\forall a \in A,, \forall c \in C, P(A = a, C = c) = P(A = a) \cdot P(C = c)$，即 A, C 相互独立。

通过上述分析可知，如果能保证在被访问的地址范围 A 内，内存地址 a 除低 b 位外完全相同，则可保证内存地址与缓存组号去相关。此时，内存地址到缓存组号的信道被阻隔。

具体到对缓存攻击的防护中，应当让查表过程中访问的地址除低 b 位外完全相同。这需要查找表的大小不大于缓存行大小 B。然而，不论是 AES 算法还是 SM4 算法，其 T 表实现中 T 表的大小都为 1024 Byte，这超过了本作品中缓存行大小 B=64Byte。因此，必须想办法将 T 表大小降低到不超过缓存行大小，故提出并行拆分表防护方案。

并行拆分表方案的本质就是通过减小查找表各表项的大小，从而让每张查找表的大小都不超过缓存行大小，实现防护。在该方案下，原查找表中的每一项都会被并行拆分成若干部分，而后各部分将被重新组合，按照原查找表的顺序编成若干项数不变但表项缩短的新查找表。每张新查找表的大小都不超过缓存行的大

小。此时，原查找表中各表项被分在不同的新查找表中，故查表时需查询每张新查找表，并将结果连接起来，恢复原来的表项。从原查找表到若干张并行查找表，表内数据的排列方式改变，但数据总量不变。

由于生成新查找表时原查找表中的各表项是被并行分割的，且查表时查找各表表项的过程在逻辑上也是并行的，故新查找表被称为“并行查找表”。各张并行查找表通过表索引编号，每张表中根据索引查询表项。生成并行查找表时，保证原查找表中的被拆分项与各张并行查找表中的拆分所得项索引相同；查询并行查找表时，原查找表中的被拆分项从各张并行查找表中索引相同的拆分所得项拼接恢复。

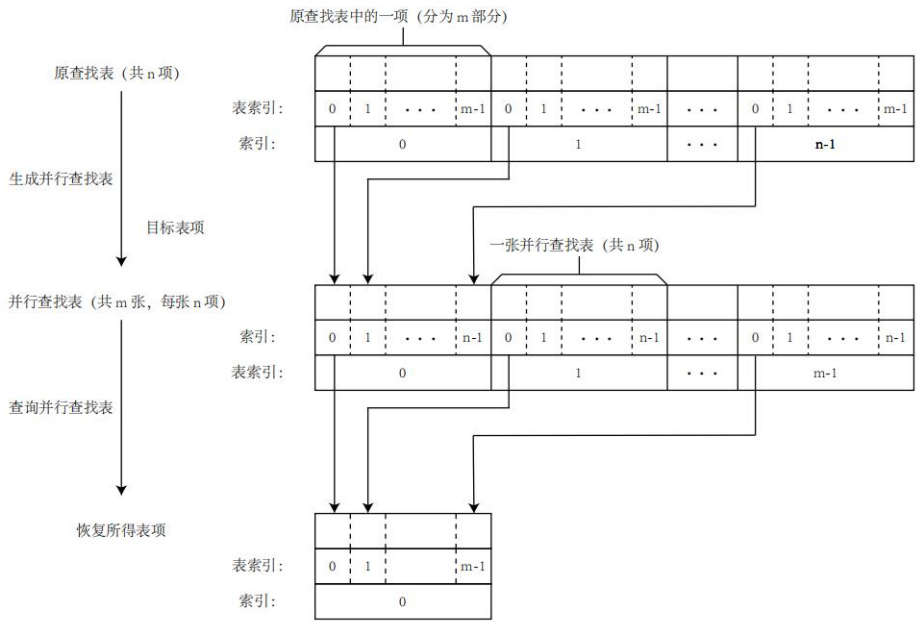

图 2-1　生成和查询并行查找表的流程

在并行拆分表方案下，由于并行查找表在查表时各表索引相同，所以各表被访问地址一一对应，其信息量较并行查找表中任意一张表的访问地址没有任何增加，故可以只考虑一张表的被访问地址在缓存攻击侧信道中的作用。又因为查询并行查找表时，各张并行查找表的大小都不超过缓存行大小 B，故并行查找表中每一张表都可以满足被访问地址除低 b 位外完全相同。因此在并行查找表的查询过程中，内存地址到缓存组号的信道能够被阻隔。由此可见，并行拆分表方案能够防御缓存攻击。

综上所述，并行拆分表防护方案通过将原查找表的各表项并行拆分为若干部分，再重新组合为若干并行查找表，从而降低查找表大小至不超过缓存行大小，

进而实现对缓存攻击的防御。

2.3 防护方案实现

2.3.1 常量时间实现

(1) 防护思路与总体方案

采用基于位运算的纯软件实现策略：通过复合域分解将 AES 中 S-BOX 的求逆和仿射变换转换为在 GF($(2^2)^2$)上的固定运算序列，所有操作均通过位运算完成，消除条件分支和基于秘密数据的内存访问。该方案确保加密过程执行固定的指令序列，执行时间与输入数据及密钥无关，从根本上防御基于时序和缓存的侧信道攻击。

常量时间实现完全避免查表操作，通过数学变换将非线性运算转换为确定的位操作序列，提供更强的侧信道防护。

(2) AES 实现方案与关键代码

a. AES S-BOX 运算

$$w_{out} = S \cdot w_{in}^{-1} + c \text{（公式 2-4）}$$

其中: w_{in}^{-1}是在 GF($(2^2)^2$)中的乘法逆元，S 是仿射变换矩阵，c = 0x63 = 01100011 是常数向量。

b. 复合域分解

采用塔式域分解：

$$GF(2^8) \rightarrow GF((GF((GF(2^2)^2)^2)) \text{（公式 2-5）}$$

c. 坐标变换

基变换映射：

$0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80 \rightarrow 1, Y, Y^2, Y^3, Y^4, Y^5, Y^6, Y^7$（公式 2-6），其中 Y = 0x41。

d. 复合域中的乘法运算

在 GF($(((2^2)^2)^2)^2$)中，令$A = a_0 \cdot 1 + a_1 \cdot x^4$, $B = b_0 \cdot 1 + b_1 \cdot x^4$有：

$$AB = (a_0b_0 + 8a_1b_1) \cdot 1 + (a_1b_0 + (a_0 + a_1)b_1) \cdot x^4$$（公式 2-7）

e. 逆元求解

给定$A = (a_0, a_1)$，求$B = (b_0, b_1)$，使得 AB = 1：

$$\{a_0b_0 + 8a_1b_1 = 1\,,\ \ a_1b_0 + (a_0 + a_1)b_1 = 0\} \quad \Rightarrow$$

$$\{b_1 \ \ = (a_0^2 + a_0a_1 + 8a_1^2)^{-1} \cdot a_1\,,\ \ \ b_0 \ \ = (a_0^2 + a_0a_1 + 8a_1^2)^{-1} \cdot (a_0 + a_1)\}$$

（公式 2-8）

f. inv8 算法

$$x_0 = a_0 \oplus a_1$$

$$[y_0, y_1]\& = mul4([x_0, a_1], [a_0, a_1])$$

$$y_1 = mul4(8, y_1)$$

$$t = inv4(y_0 \oplus y_1)$$

$$[b_0, b_1] = mul4([x_0, a_1], [t, t])$$（公式 2-9）

g. 仿射变换

$b[i] = b[i] \oplus b[(i+4) mod\ 8] \oplus b[(i+5) mod\ 8] \oplus b[(i+6) mod\ 8] \oplus b[(i+7) mod\ 8] \oplus c[i]$（公式 2-10）

SubWord/SubLong 和 InvSubLong 函数通过纯粹的位运算实现了 AES-SBOX。以 SubLong 为例，具体实现代码如下：

```
static void SubLong(u64 *w) {
    u64 x, y, a1, a2, a3, a4, a5, a6;

    // 将输入指针 w 所指向的 64 位数据读入寄存器 x
    x = *w;

    // 对输入字节做位级旋转、按位选择和线性组合。
    y = ((x & U64(0xFEFEFEFEFEFEFEFE)) >> 1) | ((x & U64(0x0101010101010101)) << 7);
    x &= U64(0xDDDDDDDDDDDDDDDD);
    x ^= y & U64(0x5757575757575757);
    y = ((y & U64(0xFEFEFEFEFEFEFEFE)) >> 1) | ((y & U64(0x0101010101010101)) << 7);
    ...

    // 对拆分后的子域值做乘法与线性组合，生成 inv8 所需中间值。
    a1 = x;
    a1 ^= (x & U64(0xF0F0F0F0F0F0F0F0)) >> 4;
    a2 = ((x & U64(0xCCCCCCCCCCCCCCCC)) >> 2) | ((x & U64(0x3333333333333333)) << 2);
    a3 = x & a1;
    a3 ^= (a3 & U64(0xAAAAAAAAAAAAAAAA)) >> 1;
    a3 ^= (((x << 1) & a1) ^ ((a1 << 1) & x)) & U64(0xAAAAAAAAAAAAAAAA);
    a4 = a2 & a1;
    a4 ^= (a4 & U64(0xAAAAAAAAAAAAAAAA)) >> 1;
    a4 ^= (((a2 << 1) & a1) ^ ((a1 << 1) & a2)) & U64(0xAAAAAAAAAAAAAAAA);
```

对应算法：
b. 复合域分解：字节被拆分为多层 GF(2^2) 子域。
c. 坐标变换：通过旋转和掩码映射实现 GF(2^8) 到 GF((GF((GF(2^2)^2))) 的基变换。
d. 复合域中的乘法运算：按位与和异或实现 AB 的乘法组合。

对应算法：
f. inv8 算法：x0 = a0 ⊕ a1、[y0, y1] = mul4([x0, a1], [a0, a1]) 的实现。

```
    // 对中间变量做 GF(2^8) 上的逆元求解组合操作。
    a5 = (a3 & U64(0xCCCCCCCCCCCCCCCC)) >> 2;
    a3 ^= ((a4 << 2) ^ a4) & U64(0xCCCCCCCCCCCCCCCC);
    a4 = a5 & U64(0x2222222222222222);
    a4 |= a4 >> 1;
    a4 ^= (a5 << 1) & U64(0x2222222222222222);
    ...

    // 合并子域结果，生成最终逆元值。
    x = a1 & a3;
    x ^= (x & U64(0xAAAAAAAAAAAAAAAA)) >> 1;
    x ^= (((a1 << 1) & a3) ^ ((a3 << 1) & a1)) & U64(0xAAAAAAAAAAAAAAAA);
    a4 = a2 & a3;
    a4 ^= (a4 & U64(0xAAAAAAAAAAAAAAAA)) >> 1;
    ...

    // 对逆元结果执行最终 S-Box 仿射变换：每个字节与自身不同位置比特做异或。加上常数向量 0x63。
    y = ((x & U64(0xFEFEFEFEFEFEFEFE)) >> 1) | ((x & U64(0x0101010101010101)) << 7);
    x &= U64(0x3939393939393939);
    x ^= y & U64(0x3F3F3F3F3F3F3F3F);
    ...
    x ^= U64(0x6363636363636363);
    *w = x;
}
```

对应算法：
e. 逆元求解：通过按位与、旋转、异或实现 (a0^2 + a0a1 + 8a1^2)^(-1) 的计算。

对应算法：
f. inv8 算法：最后一步 [b0,b1] = mul4([x0,a1],[t,t])。

对应算法：
g. 仿射变换：直接对应 b[i] = b[i] ⊕ b[i+4] ⊕ ... ⊕ c[i]。
a. AES S-BOX 运算：整体上实现 w_out = S * w_in^{-1} + c。

2.3.2 预加载表实现

(1) 防护思路与总体方案

采用查表预加载策略：在密钥扩展或加解密操作前，主动遍历所有与查表相关的数据结构（TE0~TE3、TD0~TD3、XBOX_0~XBOX_3、KBOX_0~KBOX_3 等），按 cache line 为步长访问每个表区域，使这些表的 cache line 被加载到 L1（或更高层次）缓存。之后的密码运算对表的多数访问将直接命中缓存，从而减少或消除数据相关的 cache miss 差异，降低基于缓存的侧信道能见度。与 constant-time 重写不同，预加载方案不改动加密核心逻辑，实现侵入性低，适用于已有基于查表实现的代码库。

(2) AES 实现方案与关键代码

在 AES T-box 实现文件中加入防护函数 PreloadTablesForCacheProtection()；在 AES 密钥扩展或加解密时调用预加载表函数，以降低 cache-timing 攻击风险。

a. 表预加载核心函数

核心函数 PreloadTablesForCacheProtection() 按 cache line 步长遍历 TE0~TE3、TD0~TD3、INV_S 等查表区，使用 volatile 与伪依赖变量 u 防止编译器优化掉访问。防护函数 PreloadTablesForCacheProtection()代码如下：

```
622     static void PreloadTablesForCacheProtection(void)
623     {
624         // 如果表已经预加载过，就直接返回
625         if (s_tablesPreloaded) {
626             return;
627         }
628         // 获取 CPU Cache 的行大小
629         uint32_t cacheLineSize = GetCacheLineSize();
630         // 用于累积读取，防止编译器优化掉
631         volatile uint32_t u = 0;
632         uint32_t i;
633
634         // -------------------------------
635         // 预加载加密表 TE0-TE3 到 Cache
636         // -------------------------------
637         for (i = 0; i < sizeof(TE0); i += cacheLineSize) {
638             // 每次读取一个 Cache line，确保数据被装入 CPU Cache
639             u &= *(const volatile uint32_t *)((const volatile uint8_t *)TE0 + i);
640         }
641         ...
642
643         // -------------------------------
644         // 预加载解密表 TD0-TD3 到 Cache
645         // -------------------------------
646         for (i = 0; i < sizeof(TD0); i += cacheLineSize) {
647             u &= *(const volatile uint32_t *)((const volatile uint8_t *)TD0 + i);
648         }
649         ...
651         // ------------------------------
652         // 预加载逆 S 盒到 Cache
653         // ------------------------------
654         for (i = 0; i < sizeof(INV_S); i += cacheLineSize) {
655             u &= *(const volatile uint32_t *)((const volatile uint8_t *)INV_S + i);
656         }
657
658         // ------------------------------
659         // 确保所有表元素至少被访问一次
660         // ------------------------------
661         u &= TE0[255];
662         u &= TE1[255];
663         u &= TE2[255];
664         u &= TE3[255];
665         u &= TD0[255];
666         u &= TD1[255];
667         u &= TD2[255];
668         u &= TD3[255];
669         u &= INV_S[255];
670
671         // 防止编译器将预加载循环优化掉
672         (void)u;
673
674         // 标记表已经预加载完成，避免重复操作
675         s_tablesPreloaded = true;
676     }
```

调用关系：

密钥扩展阶段：SetEncryptKey*Tbox()→PreloadTablesForCacheProtection()

加密操作：CRYPT_AES_EncryptTbox()→PreloadTablesForCacheProtection()

解密操作：CRYPT_AES_DecryptTbox()→PreloadTablesForCacheProtection()

(4) SM4 实现方案与关键代码

SM4 实现文件中加入防护函数 SM4_PreloadTables() 和 SM4_PreloadKeyTables()；在 SM4 密钥扩展和加解密时调用预加载表函数，以降低 cache-timing 攻击风险。

a. 加解密表预加载核心函数

核心函数 SM4_PreloadTables() 按 cache line 步长遍历 XBOX_0~XBOX_3 等查表区，使用 volatile 与伪依赖变量防止编译器优化掉访问。防护函数 SM4_PreloadTables() 代码如下：

```
static void SM4_PreloadTables(void)
{
    // 获取缓存行大小，如果没有相关函数，使用默认值64字节
    size_t cacheLineSize = 64;

    // 预加载所有XBOX表到缓存中
    volatile uint32_t dummy = 0;
    const uint32_t *tables[] = {XBOX_0, XBOX_1, XBOX_2, XBOX_3};
    const size_t tableSize = 256; // 每个表有256个元素

    for (int t = 0; t < 4; t++) {
        const uint32_t *table = tables[t];
        for (size_t i = 0; i < tableSize; i += cacheLineSize / sizeof(uint32_t)) {
            // 访问每个缓存行中的至少一个元素
            dummy ^= table[i];
        }
    }

    // 确保编译器不会优化掉上面的循环
    (void)dummy;

    // 内存屏障，确保预加载完成
    #if defined(__GNUC__)
    __asm__ __volatile__ ("" : : : "memory");
    #endif
}

```

b. 密钥扩展表预加载核心函数

核心函数 SM4_PreloadKeyTables() 按 cache line 步长遍历 KBOX_0~KBOX_3、CK、FK 等查表区。防护函数 SM4_PreloadKeyTables()代码如下：

```
static void SM4_PreloadKeyTables(void)
{
    // 获取缓存行大小，如果没有相关函数，使用默认值64字节
    size_t cacheLineSize = 64;

    // 预加载所有KBOX表到缓存中
    volatile uint32_t dummy = 0;
    const uint32_t *tables[] = {KBOX_0, KBOX_1, KBOX_2, KBOX_3};
    const size_t tableSize = 256; // 每个表有256个元素

    for (int t = 0; t < 4; t++) {
        const uint32_t *table = tables[t];
        for (size_t i = 0; i < tableSize; i += cacheLineSize / sizeof(uint32_t)) {
            // 访问每个缓存行中的至少一个元素
            dummy ^= table[i];
        }
    }

    // 预加载CK常数表
    const size_t ckSize = sizeof(CK) / sizeof(CK[0]);
    for (size_t i = 0; i < ckSize; i += cacheLineSize / sizeof(uint32_t)) {
        dummy ^= CK[i];
    }

    // 预加载FK常数表
    const size_t fkSize = sizeof(FK) / sizeof(FK[0]);
    for (size_t i = 0; i < fkSize; i += cacheLineSize / sizeof(uint32_t)) {
        dummy ^= FK[i];
    }

    // 确保编译器不会优化掉上面的循环
    (void)dummy;

    // 内存屏障，确保预加载完成
    #if defined(__GNUC__)
    __asm__ __volatile__ ("" : : : "memory");
    #endif
}
```

调用关系：

密钥扩展阶段：CRYPT_SM4_SetKey()→SM4_PreloadKeyTables()

加密操作：CRYPT_SM4_Encrypt()→SM4_PreloadTables()

解密操作：CRYPT_SM4_Decrypt()→SM4_PreloadTables()

2.3.3 并行拆分表

(1) 防护思路与总体方案

本方案提出了一种并行拆分表防护机制，用于在保持算法正确性的同时，显著削弱查表型密码实现的缓存侧信道泄漏。

原 T 表大小是 256 项，每项大小为 4Byte = 32bit，共 1024Byte，若希望使并行查表的大小不超过典型缓存行大小 B = 64Byte，则每项需被拆分为不少于 16 个部分，即每个部分大小不超过 2 bit。由于 2 bit 小于内存可寻址的最小单位（1Byte），因此访问拆分后的表项时，必须先通过内存地址定位目标字节，再在该字节内部通过位运算提取目标项。为此，将索引分为两部分：高位为字节索引 ，用于定位目标字节；低位为表项偏移，用于在目标字节内定位目标项。

本方案中，原 T 表将被拆分为 16 张并行查找表，每张表均有 256 项，每项均为 2 bit，总计 64Byte。因此，表索引 4 位（取值从 0 到 15），字节索引 6 位（取值从 0 到 63），表项偏移 2 位（取值从 0 到 3）。

并行拆分防护包括表生成阶段与表查询阶段。以下以 TE0 表为例，分别说明其生成与查询实现。其中，原 T 表为数组 TE0 ，新 T 表为数组 TE0_P (uint8_t TE0_P[16][64])，字节索引 Byte_index（uint8_t Byte_index = index >> 2，把 index 右移 2 位，相当于除以 4，得到目标字节在数组中的编(0~63))，表项偏移 Item_offset（uint8_t Item_offset = index & 0x03U，取 index 的最低 2 位，得到目标表项在字节内的位置(0~3))，表索引为 Table_index (uint8_t Table_index)。

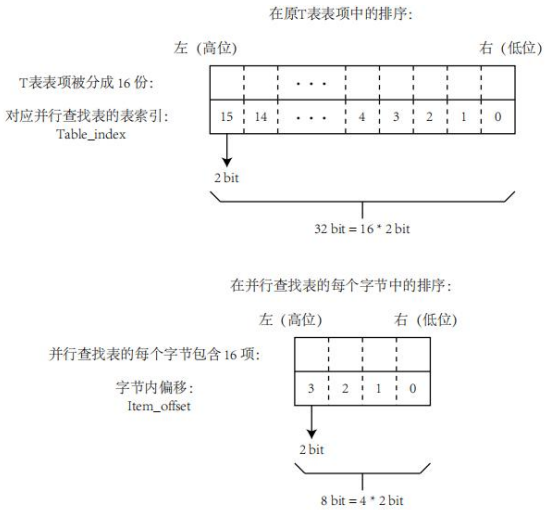

图 2-2　索引拆分方式及拆分前后各表内索引顺序

(2) 生成并行查找表

生成并行查找表需要将原 T 表中的每一项都进行拆分。原 T 表表项中拆分所得的各部分归属的并行查找表表索引从右端（低位）开始向左端（高位）依次递增。获得 Table_index 号并行查找表的表项的 C 语言代码：

TE0[index] >> Table_index >> Table_index & 0x03U

在填入同一张查找表时，先用字节索引确定目标字节，然后用表项偏移确定表项在字节内的位置。在一个字节内，表项偏移是从右端（低位）开始向左端（高位）依次递增。获得并行查找表被填入字节及被填入内容的 C 语言代码：

TE0_P[Table_index][Byte_index] // 并行查找表目标字节

(TE0[index] >> Table_index >> Table_index & 0x03U) << Item_offset << Item_offset // 目标字节要填入的 2 bit 内容

而后，再使用或运算拼接，即可将表项拼入所属字节内。并行查找表生成函数代码如下：

```c
// 宏定义：将 TE0 中的特定位填充到并行查找表 TE0_P 中
#define FILL_TE0_P(Table_index, Byte_index, Item_offset) \
    TE0_P[Table_index][Byte_index] |= \
    (TE0[index] >> Table_index >> Table_index & 0x03U) \ // 取 TE0[index] 中第 Table_index 位并右移两次，取低两位
    << Item_offset << Item_offset; // 将结果左移到对应的 Item_offset 位置，并累加到 TE0_P

// 定义并行查找表，16 个表，每个表 64 字节
uint8_t TE0_P[16][64];
uint8_t Byte_index;
uint8_t Item_offset;
uint8_t Table_index;
uint8_t index;

// 外层循环：生成 16 个并行表
for (Table_index = 0; Table_index < 16; Table_index++)
{
    index = 0; // 每个表的 TE0 索引从 0 开始
    // 内层循环：遍历 TE0 的 256 个元素
    for (int i = 0; i < 256; i++)
    {
        // 计算 TE0_P 的字节索引，每 4 个元素放到同一个字节
        Byte_index = index >> 2; // 相当于 index / 4
        // 计算当前元素在字节中的偏移（0~3）
        Item_offset = index & 0x03U; // 相当于 index % 4
        // 调用宏，将 TE0[index] 的两位数据填入 TE0_P[Table_index][Byte_index] 的对应位置
        FILL_TE0_P(Table_index, Byte_index, Item_offset)
        // 索引递增，处理下一个 TE0 元素
        index++;
    }
}
```

(3) 查询并行查找表

在通过并行查找表恢复原 T 表表项时，取出并行查找表中一张表的表项，需要先确定目标字节，而后通过位运算获得目标表项。获得目标表项的 C 语言代码：

TE0_P[Table_index][Byte_index] >> Item_offset >> Item_offset & 0x03U

需要拼接各表表项从而恢复原 T 表表项。被拼接的各项需要通过位运算被置于其所属并行查找表的表索引所对应的位置。确定各项拼接位置的 C 语言代码：

((uint32_t) (TE0_P[Table_index][Byte_index] >> Item_offset >> Item_offset & 0x03U)) << Table_index << Table_index

再使用或运算拼接，即可恢复原 T 表表项。并行查找表查询函数代码如下：

```
// 宏定义：从并行查找表 TE0_P 中取出对应元素的两位，并累加到 item
#define CONCATENATE_TE0_ITEM(Table_index, Byte_index, Item_offset) \
    item |= \
    ((uint32_t) (TE0_P[Table_index][Byte_index] \
    >> Item_offset >> Item_offset & 0x03U)) \ // 取出 TE0_P[Table_index][Byte_index] 对应元素的两位
    << Table_index << Table_index; // 将两位左移到最终结果 item 的正确位置

// 从并行查找表中读取 TE0[index] 对应的 32 位值
uint32_t ReadTE0(uint8_t index)
{
    uint32_t item = 0;
    // 计算并行表中存储该 index 元素的字节索引
    uint8_t Byte_index = index >> 2;
    // 计算元素在字节中的偏移量 (0~3)
    uint8_t Item_offset = index & 0x03U;
    // 从 TE0_P 的 16 个并行表依次取出两位并拼接到 item 中
    CONCATENATE_TE0_ITEM( 0, Byte_index, Item_offset)
    CONCATENATE_TE0_ITEM( 1, Byte_index, Item_offset)
    CONCATENATE_TE0_ITEM( 2, Byte_index, Item_offset)
    CONCATENATE_TE0_ITEM( 3, Byte_index, Item_offset)
    CONCATENATE_TE0_ITEM( 4, Byte_index, Item_offset)
    CONCATENATE_TE0_ITEM( 5, Byte_index, Item_offset)
    CONCATENATE_TE0_ITEM( 6, Byte_index, Item_offset)
    CONCATENATE_TE0_ITEM( 7, Byte_index, Item_offset)
    CONCATENATE_TE0_ITEM( 8, Byte_index, Item_offset)
    CONCATENATE_TE0_ITEM( 9, Byte_index, Item_offset)
    CONCATENATE_TE0_ITEM(10, Byte_index, Item_offset)
    CONCATENATE_TE0_ITEM(11, Byte_index, Item_offset)

    CONCATENATE_TE0_ITEM(12, Byte_index, Item_offset)
    CONCATENATE_TE0_ITEM(13, Byte_index, Item_offset)
    CONCATENATE_TE0_ITEM(14, Byte_index, Item_offset)
    CONCATENATE_TE0_ITEM(15, Byte_index, Item_offset)
    // 返回最终拼接后的 32 位值
    return item;
}
```

在具体实现时，生成并行查找表的工作被提前完成，其生成的并行查找表作为固定的查找表被写入需要防护的 C 语言程序中。之后需要获取原 T 表表项时，则使用上面给出的读取函数实现。

2.3.4 三种缓存侧信道防护方案的理论对比分析

本节针对常量时间实现、预加载防护及并行拆分表三种防护方案，从理论防护机制和算法复杂度两个维度进行系统性对比分析，旨在揭示其内在的安全特性与性能权衡关系。

(1) 防护机制与安全强度分析

1）常量时间实现方案

该方案通过严格消除数据依赖的时序特征来实现防护。具体而言，它将所有敏感操作（包括内存访问、分支判断和算术运算）转换为具有固定执行周期的指令序列，确保任何密钥或中间值的差异都不会反映在缓存访问模式或执行时间上。从安全强度来看，该方案能够同时抵御基于缓存状态、分支预测器和时序测量的侧信道攻击，具备理论上的完备安全性。然而，这种安全性的实现依赖于对算法数据流的彻底重构，通常需要通过底层汇编编程或专用指令（如 RISC-V 密码扩展指令）实现，从而导致指令数目显著增加和执行路径延长。

2）预加载防护方案

该方案基于缓存行为均一化策略，在算法执行前主动将全部敏感数据载入缓存，使后续计算阶段的缓存访问命中率保持稳定。其防护有效性依赖于预加载的完备性和缓存一致性维护能力。在实际系统中，该方案可通过硬件预取机制或软件控制的缓存加载指令实现，能够有效对抗基于缓存命中/未命中时序的观测类攻击。但由于其不改变算法核心的数据依赖关系，在面临主动缓存操纵攻击（如 Prime+Probe）时可能存在局限。该方案的优势在于实现轻量，只需在算法初始化阶段增加线性预热操作，适合对性能敏感的应用场景。

3）并行拆分表方案

作为创新型防护设计，该方案采用结构混淆与并行化相结合的方法。通过将核心查表操作分解为多个正交的子运算单元，并采用并行访问策略，使得单个密钥字节的访问模式被分散到多个独立的缓存序列中。这种设计不仅降低了敏感数

据与缓存地址间的统计相关性，还通过预置缓存预热机制维持了访问时序的稳定性。从安全视角看，该方案在保持算法结构基本不变的前提下，实现了访存模式的随机化改造，对多种缓存攻击均能形成有效干扰。特别值得关注的是，其并行化特性使其在多核并发环境下能保持更好的性能扩展性。

(2) 算法复杂度与性能特征分析

从计算复杂度理论角度分析，三种方案均保持原算法的时间复杂度量级（如 AES 的 O(n)特征），但常数因子存在显著差异：

1）常量时间方案的常数因子最大，主要源于：①条件分支的算术化转换带来的指令膨胀；②内存访问的序列化改造导致并行度下降。实测显示其性能损耗通常达 2-5 倍，但可通过硬件加速指令（如 ARM NEON、RISC-V 向量扩展）部分补偿。

2）预加载方案的复杂度增加主要体现在初始化阶段，需增加一次线性扫描的预热操作（复杂度 O(m)，m 为敏感数据结构规模）。在稳态计算阶段，由于缓存命中率提升，反而可能获得比未防护版本更优的性能表现。

3）并行拆分方案通过算法重构将原本串行的查表操作转化为并行子运算，虽然单次运算的指令数有所增加，但得益于：①缓存局部性提升减少内存停滞；②并行流水线深度优化，其实际性能损耗可控制在 20%-50%区间，且随处理器核数增加呈现超线性加速特性。

综合上述分析，三种缓存侧信道防护方案在安全性和性能上的权衡各有特点：常量时间实现方案提供了最为严格的安全保障，能够抵御广泛的时序和缓存攻击，但其高额的指令膨胀和执行路径延长导致性能损耗明显；预加载方案实现简单、开销低，通过缓存预热可有效均一化访问时序，适合性能敏感但安全要求中等的应用场景；而并行拆分表方案则在兼顾安全性与性能上表现出独特优势：通过拆分查表操作并并行执行，它不仅降低了缓存访问模式与敏感数据的统计相关性，实现对多种缓存攻击的有效干扰，同时利用并行特性和缓存局部性优化保持了较高的运行效率。在多核处理器环境下，该方案的性能扩展能力尤为突出，使其成为在保持算法结构基本不变的前提下，同时兼顾高安全性与可接受性能开销的优

选策略。

2.4 检测方案与实现

2.4.1 缓存侧信道安全检测方案

为全面评估防护后密码算法的安全性与性能表现，本项目设计并实现了完整的检测与评估体系，其中缓存侧信道安全检测是核心环节。该检测方案旨在验证防护后的 AES 与 SM4 实现能否有效抵御典型缓存攻击（如 Prime+Probe、Flush+Reload），并量化其在真实硬件环境下的安全防护能力。

(1) 检测总体方案

缓存侧信道攻击的利用路径可抽象为：受害者密钥→内存访问地址→缓存组号→时间信息。攻击者通过测量内存访问时间，推断受害者在缓存中的访问模式，从而间接恢复密钥。因此，检测的核心问题是：缓存访问状态是否与密钥存在统计相关性。若二者存在显著相关，则说明实现存在信息泄露风险。

本项目采用数据逆推检测（Data-Driven Reverse Inference）方法，从真实的缓存攻击场景出发，直接采集时间信息并反推缓存访问状态，再对其与密钥进行统计相关性分析。这种方法不依赖理论假设，能够真实反映实际攻击可行性，尤其适用于评估防护加固的有效性。

(2) 数据逆推检测原理与流程

数据逆推检测通过以下三个阶段完成：

1）时间信息采集

使用高精度计时器（如 x86 架构下的 RDTSC 指令或 RISC-V 性能计数器）记录特定内存访问序列的延迟。时间短表示缓存命中，时间长表示缓存未命中。通过多轮采样（通常 $>10^6$ 次）获得时间序列数据，并在固定 CPU 频率、关闭干扰进程的环境中运行以减少噪声。

2）缓存状态反推

应用缓存攻击原语从时间序列反推出受害者访问的缓存组号。Flush+Reload 攻击：攻击者先刷新（Flush）目标缓存行，等待受害者执行后再加载（Reload），通过加载延迟判断该行是否被访问。Prime+Probe 攻击：攻击者预先填充（Prime）

目标缓存组，在受害者执行后探测（Probe）访问时间，若时间增加则表明该组被替换。输出缓存访问状态序列 C={c0,c1,…,cn}，反映算法执行时的缓存使用模式。

3）相关性分析

将缓存访问序列与对应的密钥序列 K={k0,k1,…,kn}进行统计分析，评估密钥与缓存行为之间的相关性。

本实验硬件方面要求支持高精度计时及缓存刷新指令的处理器平台。在 Mastik 攻击框架基础上扩展实现，融合时间采集、缓存分析与可视化展示模块。已用于评估 AES 与 SM4 多种实现版本（原始实现、常量时间实现、预加载表实现、并行拆分表实现），并验证防护措施在真实攻击下的安全提升效果。

2.4.2 正确性与性能测试程序

在完成防护加固模块的设计与实现后，有必要对防护后的算法进行系统性的验证与评估。正确性测试旨在确认防护后的 AES 与 SM4 算法在功能上与标准实现保持一致，不因防护逻辑的引入而产生加密结果偏差；性能测试则用于量化防护措施对算法运行效率的影响，评估不同防护策略在安全性与性能之间的权衡关系。通过这两类测试，可以确保系统既具备正确的加密逻辑，又在性能上满足可接受的开销范围，从而验证所提出防护策略的工程可行性与优化价值。

(1) 测试目标

本节测试的目标包括两个方面：

正确性验证：检查防护后 AES 与 SM4 的密钥设置函数和单组加解密函数是否在逻辑上与标准实现完全一致。即在相同输入条件（密钥、明文）下，防护版本输出的密文应与权威实现（如 pycryptodome 的 AES 模块、gmssl 的 SM4 模块）输出结果逐字节相同；解密后明文应能正确还原。

性能评估：分析防护策略对算法执行效率的影响。由于防护通常会增加额外计算（如恒定时间逻辑、缓存预热或表项拆分），因此必须通过定量测试评估防护前后在密钥设置与加解密阶段的运行时间变化。

(2) 测试对象

测试对象是密钥设置函数和单组加解密函数——它们是开源库提供的密码算法 API 的底层实现，也是测信道攻击和安全防护的对象。由于 openHiTLS 中没有将它们作为 API 提供给用户，因此我们需要额外补充它们的函数原型:

```c
#include <stdint.h>
#include <stddef.h>

// ================== AES ================
#define CRYPT_AES_MAX_ROUNDS  14
#define CRYPT_AES_MAX_KEYLEN  (4 * (CRYPT_AES_MAX_ROUNDS + 1))

typedef struct {
    uint32_t key[CRYPT_AES_MAX_KEYLEN];
    uint32_t rounds;
} CRYPT_AES_Key;

void SetEncryptKey128(CRYPT_AES_Key *ctx, const uint8_t *key);
// void SetEncryptKey192(CRYPT_AES_Key *ctx, const uint8_t *key);
// void SetEncryptKey256(CRYPT_AES_Key *ctx, const uint8_t *key);
void SetDecryptKey128(CRYPT_AES_Key *ctx, const uint8_t *key);
// void SetDecryptKey192(CRYPT_AES_Key *ctx, const uint8_t *key);
// void SetDecryptKey256(CRYPT_AES_Key *ctx, const uint8_t *key);
int32_t CRYPT_AES_Encrypt(const CRYPT_AES_Key *ctx, const uint8_t *in, uint8_t *out, uint32_t len);
int32_t CRYPT_AES_Decrypt(const CRYPT_AES_Key *ctx, const uint8_t *in, uint8_t *out, uint32_t len);

// ================== SM4 ================
#define CRYPT_SM4_BLOCKSIZE     16
#define CRYPT_SM4_BLOCKSIZE_16  256
#define CRYPT_SM4_ROUNDS        32
typedef struct {
    uint8_t iv[CRYPT_SM4_BLOCKSIZE];
    uint32_t rk[CRYPT_SM4_ROUNDS];
} CRYPT_SM4_Ctx;

int32_t CRYPT_SM4_SetKey(CRYPT_SM4_Ctx *ctx, const uint8_t *key, uint32_t keyLen);
int32_t CRYPT_SM4_Encrypt(CRYPT_SM4_Ctx *ctx, const uint8_t *in, uint8_t *out, uint32_t length);
int32_t CRYPT_SM4_Decrypt(CRYPT_SM4_Ctx *ctx, const uint8_t *in, uint8_t *out, uint32_t length);
```

(3) 测试数据

测试程序提供两类数据来源：固定测试数据和随机测试数据。

固定测试数据：系统自带的一组标准化密钥与明文样例，便于不同算法和实现间的对比。

随机测试数据：使用 Python 脚本自动生成，脚本随机产生多组 16 字节密钥与明文块，并调用 pycryptodome（AES）和 gmssl（SM4）生成对应的参考密文。

这两种来源确保了数据的覆盖性与参考结果的可靠性，为正确性验证提供标准答案。

(4) 测试方案

测试程序设计了三种运行模式，以分别完成正确性验证与性能评估。

模式一：正确性测试

目的：验证防护后算法的加解密结果是否与标准实现一致。

步骤：逐次读取每组测试数据（密钥、明文）。调用密钥设置函数和加密函数，生成密文，比较输出密文与预期密文是否一致。对解密函数重复上述过程，验证明文是否恢复正确。

模式二：性能测试Ⅰ

目的：评估防护方案对密钥扩展阶段的性能影响。

步骤：随机生成多组密钥，对每组密钥多次调用加解密密钥设置函数，记录平均执行时间。

模式三：性能测试Ⅱ

目的：测量整体加密/解密时间，这是最符合实际应用场景的测试模式，也是衡量防护后代码性能的主要依据。

步骤：随机选定一个密钥并设置，使用该密钥对大量随机明文数据块执行加解密操作，统计总执行时间。

3. 系统测试与结果

3.1 实验环境

实验使用的 CPU 为 12th Gen Intel(R) Core(TM) i7-12700H, 操作系统为 Ubuntu 22.04 LTS。在性能测试中，本项目使用 cpulimit 和 stress-ng 工具模拟不同的系统负载情况. 实验中涉及的所有负载情况如表 3-1 所示.

表 3-1 实验负载情况说明。$(TARGET)表示测试程序；$(TIMEOUT)表示维持时间，默认 60s.

负载等级	指令	CPU	内存	Swap
Low	cpulimit -l 100 -b -z -e $(TARGET) stress-ng --vm 4 --vm-bytes 1G --vm-hang $(TIMEOUT) --timeout $(TIMEOUT)	单核 100%	1.67G/7.57G	181M/2G
Medium	cpulimit -l 50 -b -z -e $(TARGET) stress-ng --vm 4 --vm-bytes 2G --vm-hang $(TIMEOUT) --timeout $(TIMEOUT)	单核 50%	2.70G/7.57G	187M/2G
High	cpulimit -l 20 -b -z -e $(TARGET) stress-ng --vm 8 --vm-bytes 4G --vm-hang $(TIMEOUT) --timeout $(TIMEOUT)	单核 20%	4.67G/7.57G	181M/2G
Extreme	cpulimit -l 10 -b -z -e $(TARGET) stress-ng --vm 16 --vm-bytes 8G --vm-hang $(TIMEOUT) --timeout $(TIMEOUT)	单核 10%	7.45G/7.57G	1.60G/2G

3.2 安全性测试

3.2.1 AES

对 AES 的原始代码、常量时间防护代码、预加载表防护代码和并行拆分表防护代码第一字节尝试使用 PrimeProbe 进行 Cache 访问分析。结果以热力图形式呈现，横坐标为不同的 L1 Cache Set，纵坐标为不同的明文输入，明暗反映

在当前输入下某 Cache Set 在加密过程中被访问的可能性；如果热力图会呈现有规律的明亮斜条纹，则说明存在密钥泄露。所得结果如图所示：

原始代码：明亮的斜线体现出密钥的第一个字节高四位为 0000。

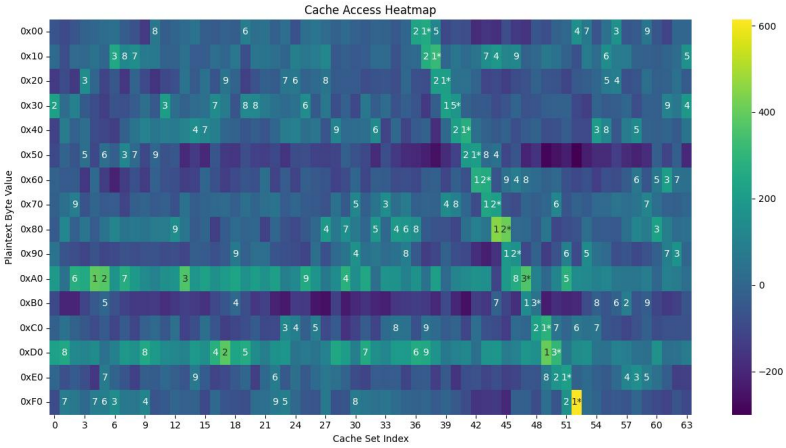

图 3-1 对原始代码进行 cache 访问的热力图

并行拆分表防护代码：热力图呈现明显的横条状，说明每个缓存组访问情况基本一致，有效隐藏了密钥信息。

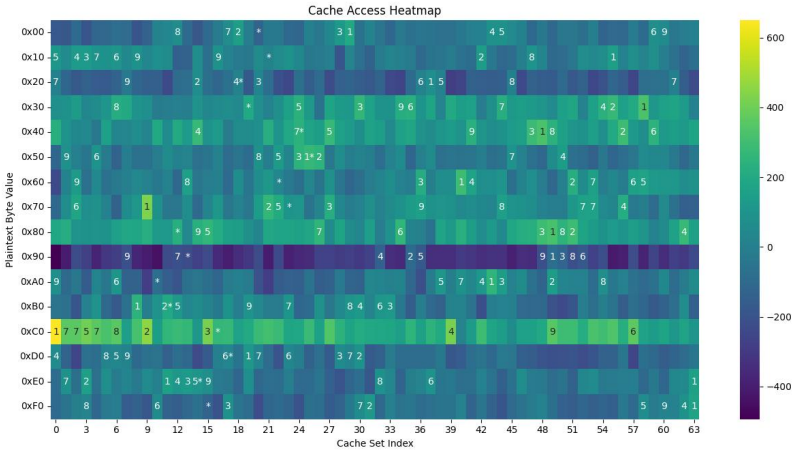

图 3-2 对并行拆分表防护代码进行 cache 访问的热力图

预加载表防护代码：明亮项分布松散，没有规律，隐藏了密钥信息。

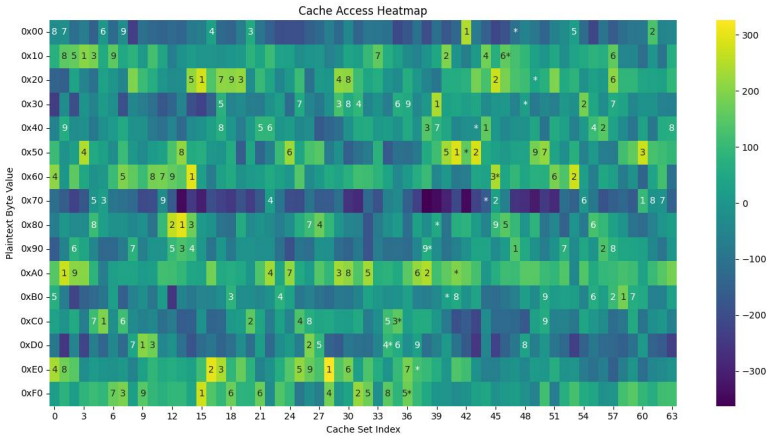

图 3-3 对预加载表防护代码进行 cache 访问的热力图

常量时间防护代码：同样分布松散，没有规律。

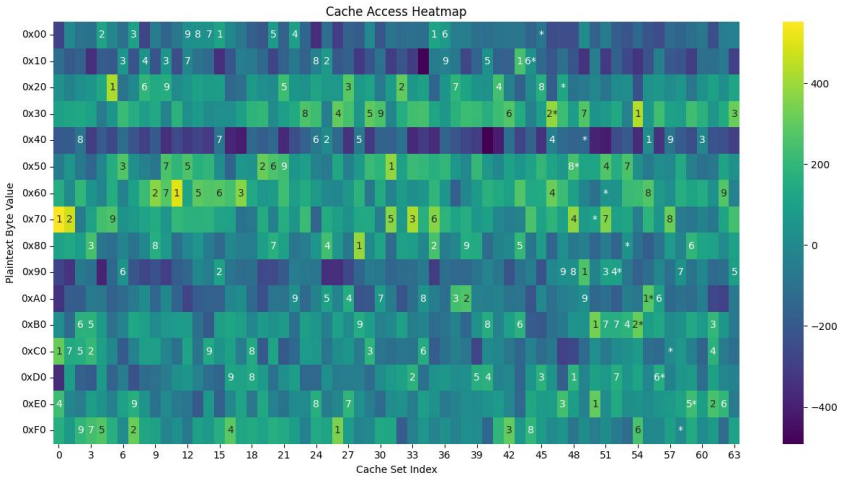

图 3-4 对常量时间防护代码进行 cache 访问的热力图

3.2.2 SM4

对 SM4 的原始代码、预加载表防护代码和并行拆分表防护代码第一字节尝试使用 PrimeProbe 进行 Cache 访问分析，所得结果如图所示：

原始代码：有规律的明亮斜线体现出密钥的第一个字节高四位为 1011。

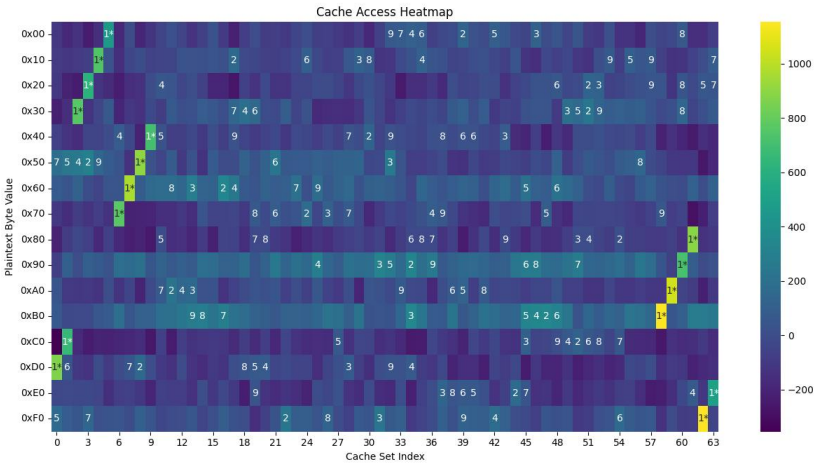

图 3-5 对原始代码进行 cache 访问的热力图

预加载表防护代码：明亮项分布松散，没有规律，隐藏了密钥信息。

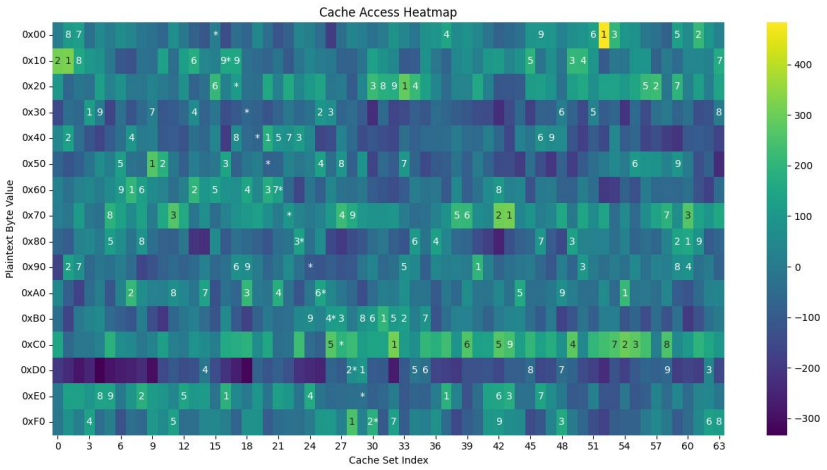

图 3-6 对预加载表防护代码进行 cache 访问的热力图

并行拆分表防护代码：热力图呈现明显的横条状，说明每个缓存组访问情况基本一致，有效隐藏了密钥信息。

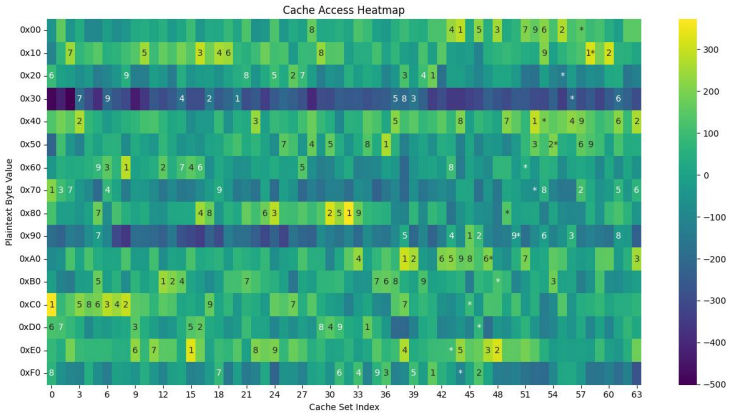

图 3-7 对并行拆分表防护代码进行 cache 访问的热力图

3.2.3 总结

从图中可以看出，三种防护代码均有效地阻止了密钥信息的泄露。然而，原始代码对应的访问时间统计图特征足够显著，评估程序可以准确地恢复密钥。

随后，针对预加载表防护代码，我们进行了进一步测试：尝试将 Prime 操作插入到预加载之后执行，测试结果如图 3-8 所示。从中可以发现明显的可以利用的统计特征，评估程序也准确地恢复了密钥。

对预加载表防护代码执行 prime 操作:两端明亮的斜线体现出密钥该字节高四位为 0111。

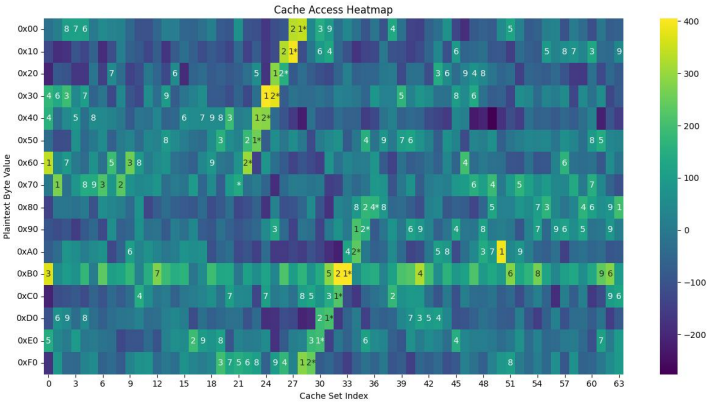

图 3-8 对预加载表防护代码执行 prime 操作的热力图

由此得出结论，对于开源库原始代码的脆弱性，常量时间和并行拆分表两种防护方案均能有效缓解基于 Cache 的测信道攻击。预加载表防护方案有一定效果，但仍存在被攻击者绕过的风险。

3.2.4 基于密钥与缓存访问相关性的安全评估

在获得不同密钥下不同明文的缓存访问情况后，除了直接从图像着手分析防护效果，也可以通过概率统计的方法检测防护情况。从前面的分析可知，若能证明密钥与缓存访问情况无关，则可以证明密码算法实现能有效抵抗缓存攻击；反之，则意味着在攻击者有能力由时间数据反推缓存访问情况的前提下，密钥信息会经由缓存访问情况被泄露给攻击者。因此，需要从概率统计上分析密钥与缓存访问情况的相关性。

要证明密钥与缓存访问情况无关，应当从两个指标入手：第一，在同一个密钥下，重复多次实验获得的缓存访问情况之间的离散程度（可用不同次结果之间的欧氏距离刻画）；第二，在不同密钥下，不同组实验获得的不同组缓存访问情况之间的离散程度（可用不同组的结果之间的欧氏距离刻画）。称第一个指标为“组内距离”，第二个指标为“组间距离”。

组内距离分析的是同一个密钥下的缓存访问情况，反映了在一个密钥下得到的缓存访问结果是否会集中分布，该指标表示的离散程度越大，即密钥相同时缓存访问情况尽可能随机，则越安全；组间距离分析的是不同密钥下的缓存访问情况，反映了不同密钥下访问结果是否会有显著差别，该指标表示的离散程度越小，即密钥不同时缓存访问情况尽可能一致，则越安全。对于一种密码算法的实现，组内距离越大，组间距离越小，该实现越安全。

在具体进行数据处理时，本工作在对每一种密码算法实现的检测中，选择了三个密钥，每个密钥选择了四次实验的结果，且每个结果均为该次实验中对第一个字节进行攻击的检测结果，实验结果均使用向量表示（每个向量均有 64 个元素）。而后，对实验结果归一化，以便使用欧氏距离刻画离散程度。

这之后，把同一个密钥下（组内）的实验结果向量重复循环移动（即将向量内的各个元素进行首尾相接的循环移动），实现对组内 3 个结果向量的所有循环移动距离的依次遍历（需要 3 层嵌套循环）。进行该操作的原因是，由于缓

存攻击的特点，实验结果向量各元素的索引并不直接是缓存组号，而是随受害者、攻击者双方地址影响的缓存组相对位置，即与缓存组号之间可能存在一定的偏移量。为了对齐不同次实验的实验结果向量，就必须使用循环移动的方法，且一个向量不移动，另外 3 个向量遍历循环移动距离。在遍历过程中，求组内任意两向量之间的欧氏距离并取平均（组内任意两向量组合，共 6 对，每对求欧氏距离，再取平均），获得平均欧氏距离，并选择使平均欧氏距离最小的循环移动距离组合，即为恰能对齐的循环移动距离组合，并记录该距离组合和平均欧氏距离。该最小的平均欧氏距离即为组内距离。在完成遍历之后，根据该距离组合对齐实验结果向量，并求出对齐后实验结果的平均值，即组内平均值。

使用上一段的方法处理每一个密钥下（每一组）的实验结果，一方面使实验结果向量对齐，另一方面获得了组内距离和组内平均值。之后，使用各组的组内平均值，求出任意两组组内平均值之间的欧氏距离，即组间距离。为综合考察每种密码算法实现的安全性，对求得的各组组内距离取平均，获得平均组内距离；对求得的任意两组之间的组间距离取平均，获得平均组间距离

通过上述流程，获得不同密码算法实现的平均组内距离、平均组间距离，其结果如下表所示：

表 3-2 AES 密码算法不同实现的组内组间距离

数据	AES 并行拆分表	AES 预加载表	AES 无防护	AES 常量时间
平均组间距离 avg_between	2.615042424905355	2.591707922113259	3.053884042991195	3.243475669879215
平均组内距离 avg_within	4.424904593527352	4.134386096365897	3.890126811103903	4.042066385413118
avg_between / avg_within	0.590982781579195745649074814832943	0.626866446844757976299764443079298	0.785034573750718676169032238108 7	0.802430084172830003752323037245 18

表 3-3 SM4 密码算法不同实现的组内组间距离

数据	SM4 并行拆分表	SM4 无防护	SM4 预加载表
平均组间距离 avg_between	2.994897958633402	2.064422565574546	3.169059365423113
平均组内距离 avg_within	4.471777341617094	1.604170045096175	3.509754825127729
avg_between / avg_within	0.669733246948825136440835528007 83	1.286910057874056253920364558 2219	0.902928986017643777594907182136674

根据上述结果，综合平均组内距离和平均组间距离考虑，可认为在对 AES 和 SM4 密码算法的实现中，并行拆分表实现面对缓存攻击时的安全性明显优于无防护情况。

3.3 性能测试

在不同系统负载情况下，对上述四种实现进行性能测试，所得结果如图 3-9，3-10 所示。横轴是四种不同的负载情况，纵轴是时间（CPU 周期数）。

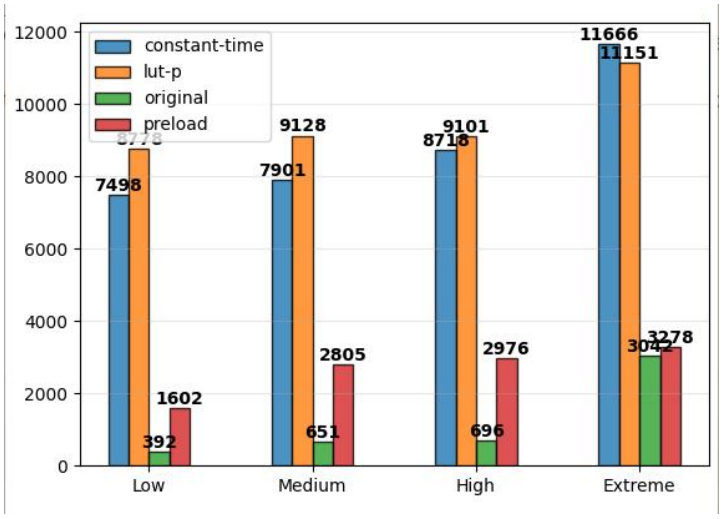

图 3-9　AES 测试结果

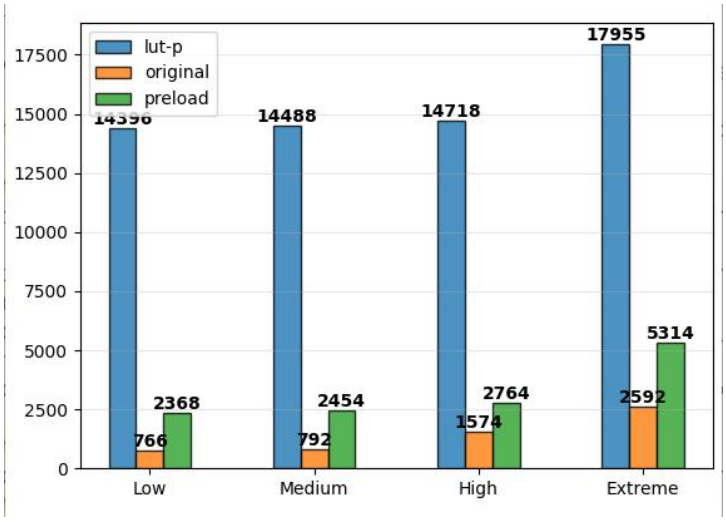

图 3-10　SM4 测试结果

从图中可以看出，预加载表方案性能最好，常量时间防护方案和并行拆分表方案性能接近。后续，我们会对并行拆分表实现做优化。

在本次测试中，我们从安全性和性能两个方面对开源库的防护方案进行了评估。从安全性角度看，常量时间和并行拆分表两种防护方案都能有效缓解基于 Cache 的侧信道攻击，对于原始代码中的脆弱性提供了有效的防护。相比之下，预加载表方案虽然能提供一定的防护效果，但仍然存在被攻击者绕过的风险，因此我们认为该方案不够安全。

从性能角度来看，预加载表方案表现最好，其次是常量时间防护方案和并行拆分表方案，两者性能相近。尽管预加载表的性能优秀，但由于其安全性问题，我们将其排除在最终方案之外。最后，针对常量时间和并行拆分表方案，我们提出的并行拆分表策略与 OpenSSL 的常量时间方案性能相当。由于并行拆分表方案本身是并行实现的，我们计划进一步优化它，借助现代处理器的并行计算能力，这一优化可能使其性能超越常量时间防护方案。

4. 应用前景

(1) 多维度防护策略的设计实现与应用前景

缓存侧信道攻击（Cache Side-Channel Attack）已成为现代密码算法软件实现所面临的主要且重要威胁之一，其通过利用加密运算过程中产生的缓存访问模式泄漏，间接推断出密钥或敏感中间值，对各类密码库的安全性构成严峻挑战。国际主流开源密码库（如 OpenSSL、Crypto++ 等）已高度重视此类微架构层面的安全问题，形成了多种针对性防护方案，并在学术界与工业界广泛应用。例如，OpenSSL 采用“常量时间编程”策略，力求消除数据依赖性的时间差异，以实现高强度的安全防护；Crypto++ 则引入预加载机制，以较低性能开销削弱部分缓存攻击效果，体现出轻量级工程的折中思路。

在我国积极推进密码技术自主可控的背景下，华为推出的 openHiTLS 开源密码库，不仅集成支持国际通用密码算法，还重点实现了中国国家标准密码算法（如 SM2、SM3、SM4），是对全球开源密码库生态的重要补充。然而，与诸多现有开源密码库类似，openHiTLS 在应对处理器微架构安全漏洞方面仍存在防护不足，尤其是针对缓存侧信道攻击的防御机制有待加强。

针对该问题，我们的研究工作聚焦于为 openHiTLS 提供系统性的缓存攻击防护能力。除对已有防护策略进行适应性改造与集成之外，本项目自主提出了一种创新的并行拆分表防护策略，该策略在安全性、性能及可移植性等多个维度均展现出显著优势：

其一、高效的安全性提升

该策略通过将传统查表操作分解为多个低关联、等规模的并行子表访问，并在运行时动态重组还原原表项，从根本上削弱密钥与缓存访问模式之间的统计相关性。相较于预加载等方案，在面对主动缓存驱逐（Flush）或跨进程竞争（Prime+Probe）等复杂攻击模式时，本方法仍可保持稳定的防护强度，具备更高的抗泄漏鲁棒性，尤其适用于云环境或多线程安全模块等高并发共享硬件场景。

其二、性能与可行性的平衡

与严格的“常量时间”实现相比，本策略在维持高安全水平的同时显著降低了性能开销。由于其拆表结构与数据路径固定，部分保留编译优化空间与缓存局部性，使其在嵌入式设备或国产软硬件平台等资源受限环境中，具备更优的工程

可部署性，为实现“安全-性能”双目标提供可行的技术路径。

其三、推动国产密码库防护体系自主化

本项目所提出的防护策略具备完全自主知识产权，已成功集成至 openHiTLS 中，为其构建起持续可演进的安全能力。该方案不依赖于国外密码库实现，可覆盖 AES、SM4 等核心算法，为我国密码技术在金融、政务、通信等关键领域的落地提供了重要安全保障。此外，该策略具有良好的可扩展性与迁移能力，可推广至其他国密算法及未来自研密码标准，为国产密码生态的全面安全建设提供重要支撑。

(2) 为国产开源加密库提供 cache 类安全检测技术

为本项目构建的安全性测试平台，以国际标准加密算法（如AES）与国产商用密码算法（如SM4）为核心对象，系统设计并实现了多种类型的缓存侧信道攻击向量，作为评估密码实现安全性的主体内容。在攻击方法设计上，充分考虑了微架构攻击对底层硬件（处理器型号、缓存结构）和软件环境（操作系统、内存管理策略）的强依赖性，不仅集成如 Mastik等经典缓存攻击工具作为基础检测手段，更以“最大化攻击者能力”为原则，模拟具备高级威胁能力的攻击场景，对防护机制进行严格的压力测试与有效性验证。

在测试实施过程中，我们采用满足统计显著性的样本量进行数据采集与分析，通过重复测量与统计拟合方法有效抑制实测环境中的噪声干扰，确保攻击结果具备可复现性与置信度。这一严谨的测试方法论不仅适用于本项目中 openHiTLS 密码库的防护效果评估，也可扩展为其他敏感运算模块及开源代码的安全性检测平台，具备良好的通用性与准确性。

该测试平台的应用价值体现在三个方面：

其一，为国产密码算法实现提供了专有的安全评估工具链，推动其走向成熟与应用；

其二，所提出的检测方法具备高可移植性，可服务于各类涉及关键数据操作的代码安全验证；

其三，通过系统性实证分析，为后续防护策略的优化与标准化提供了数据支撑与实践依据，对构建自主可控的软件安全开发生态具有重要推动意义。

5. 结论

(1) 工作总结

本工作以国产开源加密库 openHiTLS 为主要研究对象，结合国际上被广泛使用的开源加密库 OpenSSL ，分析了目前国产开源加密库对 AES, SM4 等密码算法的实现面对缓存攻击的漏洞，并基于现有防护方案和创新防护方案给出了针对性防护。本工作使用现有缓存攻击手段，结合 Mastik 侧信道攻击实验开源工具包，通过对实际开源加密库进行代码分析与测试，找出了国产开源加密库中现存的漏洞，即在 AES，SM4 的 T 表实现中，缓存读取时间会泄露密钥信息。之后，本工作分别使用现有的预加载防护，和在本工作中创新提出的并行拆分表防护，对国产开源加密库 openHiTLS 中的 AES 和 SM4 密码算法进行了针对性防护。最后，提出并完成在完美攻击者能力的假设前提下，我们提出的并行拆分表防护方案的安全性验证。

(2) 团队心得

在本工作中，本团队开展了攻防两方面的工作，完成了对国产开源加密库面对缓存攻击的漏洞检测与防护的任务。在检测中，面对密码算法不同、加密库实现不同的情况，本团队有针对性地调整检测代码，使之能够适配对 openHiTLS 开源加密库中的 AES, SM4 加密算法检测的任务。在防护中，本团队一方面查询现有针对缓存攻击的防护的方案，并根据 openHiTLS 已有代码的具体情况，决定选择更便于实现的预加载防护；另一方面创新提出了并行拆分表防护方案，为针对缓存攻击的防护提供了新思路，并且通过实现该防护，在实际测试中检验了防护效果。

团队所有成员均为信息安全专业本科生，我们在学习《计算机组成原理》《密码学》《程序设计》等理论刻成学习基础上，查阅重要参考文献，并对多种类型 cache 攻击进行复现实验，极大锻炼了我们对理论问题的分析能力、代码编写与分析能力、以及创新性思维的训练。

(3) 未来展望

未来，本团队的工作将从以下几个方向持续深化与拓展：

首先，基于本次在国产开源加密库 openHiTLS 中所发现的缓存攻击漏洞及已验证的多类防护方案，我们将进一步推动这些防护策略在更多国密算法实现中的迁移与应用，系统性提升 openHiTLS 乃至国产密码生态的整体安全性。

其次，本次构建的检测方法与工具链，为开源加密库的缓存攻击检测提供了可行路径，后续我们将致力于对其进行通用化与模块化改造，以增强其对不同算法与硬件平台的适应性，形成标准化评估能力。

最后，团队所提出的并行拆分表防护方案展现出了良好的潜力，未来将围绕其展开更深入的理论优化与实战验证，探索在复杂微架构环境下的鲁棒性提升路径，为防御技术演进提供新的思路。

综上，本作品不仅是对现有 openHiTLS 开源库的阶段性防护实践，更为后续构建自主可控、持续演进的安全密码库体系提供一定技术支持。

参考文献

[1] WoTrus Digital Certificate. OpenSSL: How critical is it and what is it used for. Juejin Tech Community, 2025-09-25. https://juejin.cn/post/7553477399478534159

[2] openHiTLS Community. Overview of domestic and international open-source cryptographic libraries: evolution, national algorithm adaptation, and maintenance challenges. CSDN Blog, 2025-10-02. https://blog.csdn.net/openHiTLS/article/details/151854920

[3] Huawei ICT Open Source Industry and Ecosystem Development Dept. Fully self-developed cryptographic suite openHiTLS and full-scenario smart front-end framework openInula released. openInula Official Website, 2023-09-21. https://openinula.net/news/4

[4] Ju Li Mao. Computer Architecture—Understanding Cache (Part I). Zhihu Column, 2021-03-12. https://zhuanlan.zhihu.com/p/386919471

[5] Wang Z G. Software implementation of AES algorithm against side-channel attacks in smart cards [D]. Chongqing: Chongqing University, 2014.

[6] Osvik D A, Shamir A, Tromer E. Cache attacks and countermeasures: The case of AES [C] // Cryptographers' Track at the RSA Conference. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006: 1–20.

[7] The OpenSSL Project Authors. aes_core.c source code. GitHub, 2022. https://github.com/openssl/openssl/blob/master/crypto/aes/aes_core.c

[8] Crypto++ Project (Wei Dai). rijndael.cpp source code. GitHub, 2022. https://github.com/weidai11/cryptopp/blob/master/rijndael.cpp

[9] openHiTLS Project. openHiTLS source code. GitHub, 2025. https://github.com/openHiTLS/openhitls

[10] Yarom Y, Falkner K. Flush+Reload: A high-resolution, low-noise, L3 cache side-channel attack [C] // Proceedings of the 23rd USENIX Security Symposium. San Diego, CA: USENIX Association, 2014. ISBN 978-1-931971-15-7. https://www.usenix.org/conference/usenixsecurity14/technical-sessions/presentation/yarom

[11] Lou X, Zhang T, Jiang J, Zhang Y. A survey of microarchitectural side-channel vulnerabilities, attacks, and defenses in cryptography [J]. ACM Computing Surveys, 2021. https://dl.acm.org/doi/10.1145/3432893